
ResourceHive

ResourceHive
Software Architecture Document

Version 1.0

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

Revision History

Date	Version	Description	Author
01 August 2026	1.0	Initial ResourceHive software architecture baseline	ResourceHive Team

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

Table of Contents

1. Introduction.....	5
1.1 Purpose.....	5
1.2 Scope.....	5
1.3 Definitions, Acronyms, and Abbreviations.....	6
1.4 References.....	8
1.5 Overview.....	8
2. Architectural Representation.....	8
3. Architectural Goals and Constraints.....	11
3.1 Architectural Goals.....	11
3.2 Architectural Constraints.....	11
3.3 Significant Architectural Decisions.....	12
4. Use-Case View.....	13
4.1 Use-Case Realizations.....	16
4.1.1 Sign up.....	16
4.1.2 Verify email.....	16
4.1.3 Log In.....	17
4.1.4 Discover Organizations & Request Membership.....	17
4.1.5 Find Resources.....	18
4.1.6 Book a Resource Slot.....	19
4.1.7 Cancel Own Booking.....	19
4.1.8 View Point History.....	20
4.1.9 View and Manage Notifications.....	21
4.1.10 Create and Manage Root Tenants.....	21
4.1.11 Create Child Organization.....	22
4.1.12 Approve/Reject Membership.....	23
4.1.13 Create and Manage Resources.....	23
4.1.14 Manage Email Allowlist.....	24
4.1.15 Configure Organization Rules.....	25
4.1.16 View Bookings.....	25
4.1.17 Complete Resource Return.....	26
4.1.18 Allocate Semester Points.....	27
4.1.19 Rate Completed Resource Use.....	28
4.1.20 Open Booking Dispute.....	28
4.1.21 Review and Resolve Dispute.....	29
4.1.22 View Tenant Analytics.....	30
5. Logical View.....	31
5.1 Overview.....	31
5.2 Architecturally Significant Design Packages.....	31
5.2.1 Identity Package.....	31
5.2.2 Resource & Organization Package.....	31
5.2.3 Booking & Ledger Package.....	31

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

5.2.4 Notification Package.....	32
5.3 Domain Class Diagrams.....	32
6. Process View.....	38
6.1 Concurrency and Failure Handling.....	50
7. Deployment View.....	50
7.1 Deployment Security Boundaries.....	52
7.2 Evolution.....	52
8. Implementation View.....	52
8.1 Overview.....	52
8.2 Layers.....	53
9. Data View.....	53
10. Size and Performance.....	58
11. Quality.....	59
12. References.....	59

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

Software Architecture Document

1. Introduction

ResourceHive is a multi-tenant software platform intended to replace fragmented resource-management processes such as spreadsheets, noticeboards, email conversations, and informal messaging. It provides one verified and auditable workflow through which institutional users can discover, reserve, use, return, and review shared resources. The platform organizes customers as root organizations, each representing an independent tenant. Faculties, departments, clubs, laboratories, and other internal groups are represented as child organizations inside that tenant. Users may belong to more than one organization, but all access to tenant-owned information must be evaluated against the appropriate root organization, approved memberships, resource permissions, and administrative authority.

1.1 Purpose

The purpose of this document is to describe the significant architectural decisions that shape ResourceHive and explain how the system satisfies its functional and non-functional requirements.

The document is intended for:

- Project developers, who must understand service boundaries and dependency rules.
- Testers, who must identify required functional, authorization, database, and concurrency tests.
- Lecturers and reviewers, who must evaluate the consistency of the proposed design.
- Deployment operators, who must understand containers, gateway routing, configuration, migrations, health checks, and recovery.
- Future maintainers, who must understand why particular technical and database decisions were made.

The document connects user requirements to software services, domain classes, runtime interactions, persistent entities, deployment nodes, security boundaries, and quality controls. Particular attention is given to multi-tenancy, inherited organization administration, atomic booking and point deduction, database concurrency, append-only audit records, notification reliability, and provider-independent deployment.

1.2 Scope

The ResourceHive target architecture includes:

- A public marketing website.
- An authenticated web application.
- Institutional registration and email verification.
- Password login, account recovery, OAuth linking, access tokens, and refresh tokens.
- Root-tenant onboarding.
- Parent and child organization management.
- Organization email-domain and allowlist management.
- Membership requests, approvals, rejection, and suspension.
- Organization-administrator authorization.
- Resource creation, modification, activation, and access configuration.
- Resource availability-slot management.
- Resource discovery based on approved memberships and allowed organizations.
- Atomic booking creation and point deduction.
- Booking cancellation and policy-based refund.
- Booking completion and resource return.
- Joining bonuses and semester-based point allocations.
- Append-only point-history and balance calculation.
- Persistent recipient-scoped notifications.
- Real-time notification delivery.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

- Resource ratings after completed bookings.
Booking disputes and append-only dispute histories.
- Tenant-scoped administrative analytics.
- API gateway routing, health endpoints, and OpenAPI documentation.
- Containerized backend deployment and frontend hosting.
- Shared PostgreSQL persistence managed through committed migrations.

The architecture uses one shared PostgreSQL database. It does not use one database per tenant and does not require PostgreSQL Row-Level Security for the initial architecture. Tenant isolation is instead achieved through application authorization, tenant-scoped Prisma queries, composite database relationships, constraints, and integrity tests.

This document describes the approved target architecture of ResourceHive. Some later-phase capabilities, including optional OAuth linking, booking cancellation and refunds, semester allocations, ratings, disputes, analytics, and complete notification delivery, are represented as intended final behavior and are not all implemented in the current repository baseline.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
API	Application Programming Interface.
API gateway	The public Nginx entry point that routes requests to internal backend services.
Approved membership	An organization membership whose status is APPROVED. Only approved memberships grant normal organization access.
Booking	A reservation connecting one user to one resource-availability slot.
Child organization	An organization whose parent_id refers to another organization in the same tenant.
CI	Continuous Integration - Automated installation, validation, build, and testing performed by GitHub Actions.
CORS	Cross-Origin Resource Sharing - The browser-origin policy used between the web application and API.
DTO	Data Transfer Object used to validate and describe request data.
Identity Service	The backend service that owns signup, verification, login, password hashing, JWT creation, and user identity.
JWT	JSON Web Token used to carry signed authentication claims.
Membership role	An organization-level role: ADMIN or MEMBER.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

Term	Definition
Membership status	PENDING, APPROVED, REJECTED, or SUSPENDED.
Organization	A root tenant or an internal unit such as a faculty, department, club, office, or team.
Organization admin	A user with an approved ADMIN membership in an organization.
Platform admin	A ResourceHive operator whose platform role is PLATFORM_ADMIN.
Points ledger	The append-only history of joining bonuses, booking deductions, and booking refunds.
Prisma	The ORM and migration tool used to access and manage PostgreSQL.
Resource	A shareable facility or item owned by an organization.
Resource owner organization	The organization responsible for a resource.
RLS	PostgreSQL Row-Level Security. It is not part of the initial ResourceHive authorization design.
Root organization	The top-level organization represents one tenant. Its root_organization_id refers to its own ID.
SaaS	Software as a Service.
Service-local authentication	JWT verification performed inside the Resource, Booking, or Notification Service without contacting Identity Service on every request.
SMTP	Simple Mail Transfer Protocol used by the production email transport.
Tenant	One top-level customer using the shared ResourceHive installation. The tenant boundary is represented by root_organization_id.
Tenant isolation	Prevention of access between different root organizations.
UUID	Universally Unique Identifier used for database record identifiers.
Verification token	A single-use, time-limited token used to prove control of a signup email address.
Booking Service	The backend service that owns resource slots, booking

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

Term	Definition
	creation, cancellation, completion, point deductions, refunds, semester allocations, and booking disputes.
ERD	Diagram showing database entities and relationships.
Notification Service	The backend service that owns persistent recipient scoped notifications and approved notification delivery mechanisms.
OAuth 2.0 / OIDC	Standard protocols used for approved external account linking or authentication-provider integration.
ORM	Object-Relational Mapping, a technique used to access relational data through application-level models. ResourceHive uses Prisma as its ORM.
PostgreSQL	The shared relational database used as the authoritative persistent data store for ResourceHive.
Redis	An optional in-memory data store that may support caching, rate limiting, notification pub/sub, or WebSocket coordination.
Resource Service	The backend service that owns organizations, memberships, organization hierarchy, resources, and resource-access rules.

1.4 References

The complete list of standards, technical documentation, and diagramming tools referenced by this architecture is provided in Section 12. PlantUML and draw.io were used to prepare the design diagrams.

1.5 Overview

Section 2 explains the architectural views used to describe ResourceHive. Section 3 presents the goals, constraints, and significant architectural decisions. Section 4 describes architecturally important use cases. Section 5 presents the logical organization of the domain. Section 6 explains runtime processes, transactions, and failure handling. Section 7 maps the applications and services to their deployment environment. Section 8 describes the implementation structure and dependency rules. Section 9 describes persistent entities and database integrity. Sections 10 and 11 explain performance and quality characteristics. Section 12 lists the referenced project and external sources.

2. Architectural Representation

ResourceHive is described through several complementary views because no single diagram can communicate every relevant aspect of the system. The use-case view describes the goals of visitors, approved members, organization administrators, and platform administrators. The logical view describes the domain entities and relationships. The process view describes how software components collaborate at runtime. The deployment view maps executables to infrastructure. The implementation view describes code organization and dependencies. The data view describes persistent entities and database integrity controls.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

View	Purpose	Principle elements
Use-case view	Describes architecturally significant user goals	Actors, use cases, preconditions, flows and outcomes
Logical view	Describes the static domain structure	Users, organizations, memberships, resources, bookings, points, ratings and disputes
Process view	Describes runtime behaviour	Gateway routing, JWT validation, authorization, transactions and notifications
Deployment view	Describes physical execution	Browser, Vercel, Nginx, Docker containers, PostgreSQL and SMTP
Implementation view	Describes source-code organization	Applications, services, shared packages and layers
Data view	Describes persistent integrity	Tables, keys, indexes, constraints, triggers and transactions

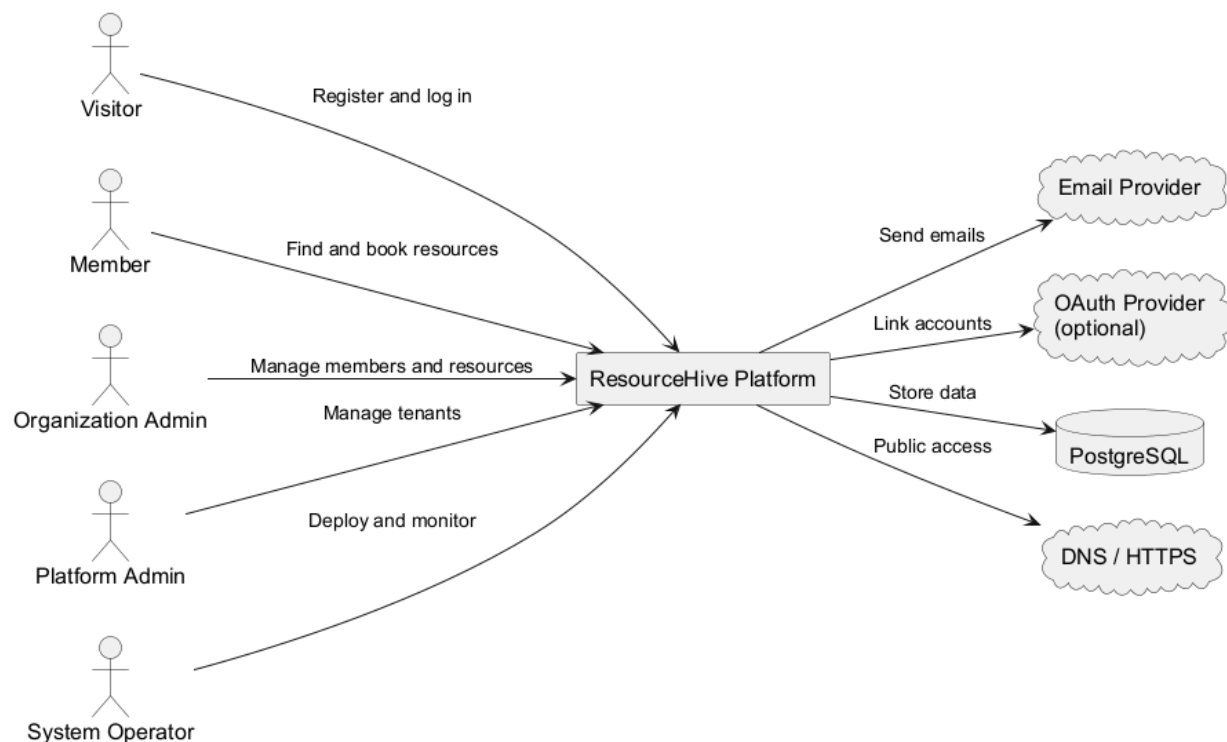


Figure 1. ResourceHive System Context

Figure 1 shows ResourceHive as a multi-tenant platform and identifies the people and external systems that interact with it. Visitors, members, organization administrators, platform administrators, and system operators use the platform. Email, optional OAuth, PostgreSQL, and public DNS and HTTPS infrastructure remain outside the ResourceHive system boundary.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

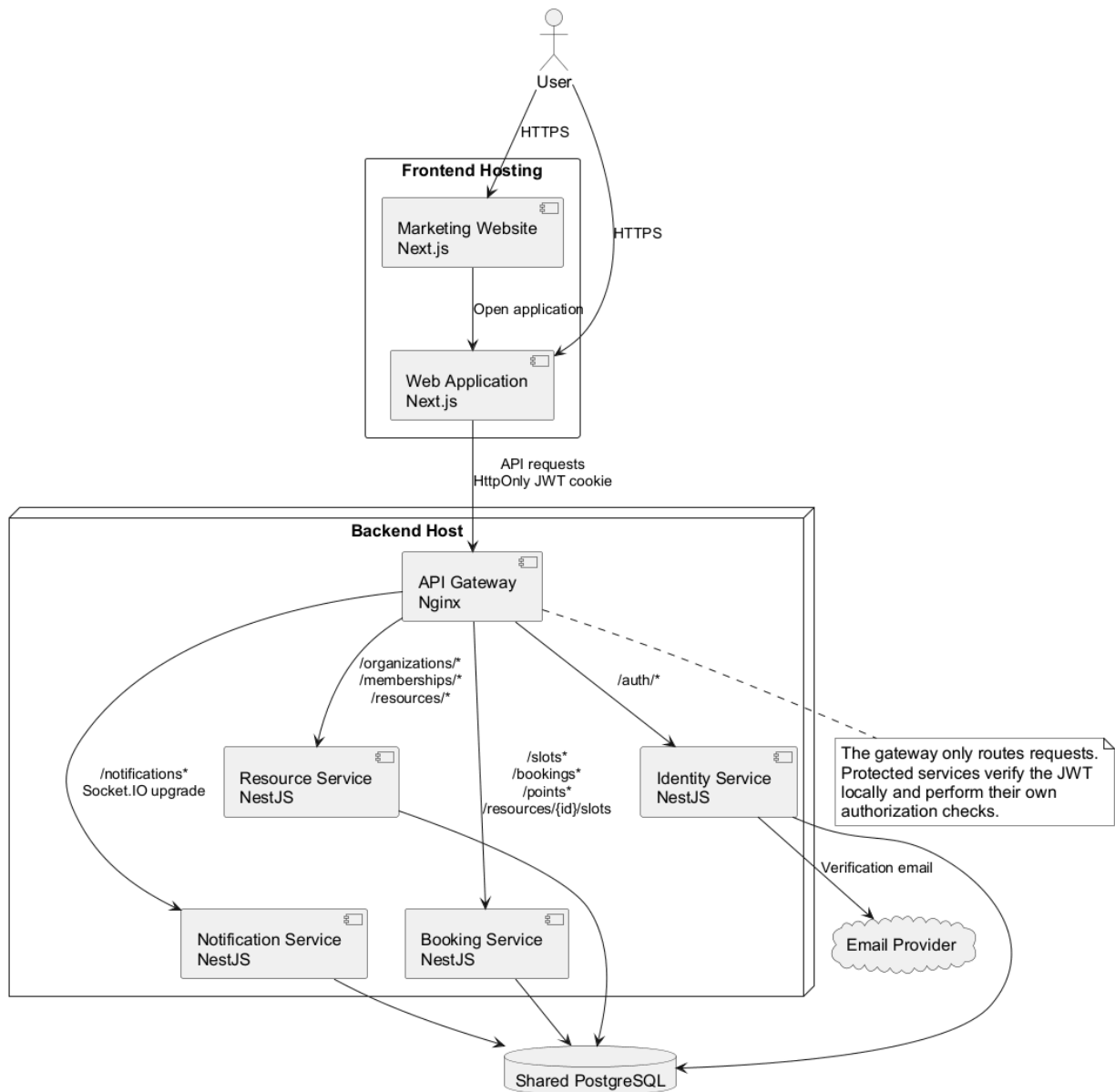


Figure 2. ResourceHive Container and Service Architecture

Figure 2 shows the main ResourceHive runtime containers. The public marketing website links to the authenticated Next.js application. Application requests pass through the Nginx API Gateway to the Identity, Resource, Booking, or Notification Service. Nginx only routes requests; protected services verify the JWT locally and perform their own authorization checks. All four services use the shared PostgreSQL database, while the Identity Service sends verification email through the configured email provider.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

3. Architectural Goals and Constraints

3.1 Architectural Goals

Tenant isolation

All tenant-owned data must remain inside the correct `root_organization_id` boundary. Knowing another tenant's object identifier must not allow a user to access that object. Authorization must therefore apply to direct object requests as well as data displayed through the frontend.

Correct concurrent booking

Concurrent attempts to book the same slot must produce no more than one active booking. Application checks alone are insufficient because two requests may observe the same slot as available. ResourceHive therefore combines application validation, database transactions, locking, and database uniqueness.

Atomic booking and points processing

A confirmed booking and its negative BOOKING point transaction must either both commit or both roll back. The system must never create a booking without its deduction or deduct points without creating the corresponding booking.

Auditable point history

ResourceHive maintains point balances through an append-only ledger rather than a mutable balance field. Joining bonuses, semester allocations, booking deductions, and refunds are separate immutable events. The current balance is calculated from the sum of these transactions.

Understandable service ownership

Identity, organization/resource, booking/points, and notification responsibilities are placed in separate services. The shared database does not permit one service to take over another service's business logic.

Provider independence

Database, frontend-hosting, backend-hosting, SMTP, OAuth, and DNS providers are selected through configuration. Vendor-specific APIs must not appear inside core domain rules unless an approved adapter isolates them.

Maintainability and verification

Every feature must include validation, authorization, tests, API documentation, container readiness, configuration documentation, and health checks where applicable. A feature is not considered complete merely because a page or endpoint exists.

3.2 Architectural Constraints

The approved architecture contains:

- Two Next.js frontend applications: the marketing website and main authenticated application.
- Four independent NestJS services: Identity, Resource, Booking, and Notification.
- One Nginx API Gateway.
- One shared PostgreSQL database.
- Prisma and committed versioned migrations.
- Docker Compose for backend orchestration.
- GitHub Actions for continuous integration.
- Vercel for hosting the frontend applications.
- One Docker-capable backend host for the initial deployment.

The tenant boundary is `root_organization_id`. Child organizations use `parent_id`. Users are global and are connected to organizations through memberships.

No service may trust a tenant identifier, organization identifier, user identifier, or role supplied by the frontend without checking authoritative data. Nginx must not contain organization-authorization rules.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

Protected services validate access-token signatures and expiry locally through the approved shared authentication package. They must not call the Identity Service for every protected request.

Redis must not replace PostgreSQL transactions, constraints, or authoritative records. It may only support justified functions such as caching, rate limiting, pub/sub, or WebSocket coordination.

3.3 Significant Architectural Decisions

Decision	Rationale	Consequence
One shared PostgreSQL database	Supports atomic booking and point changes and central tenant integrity	Services require strict logical ownership despite physical sharing
Local JWT validation	Removes a network call to Identity Service from each request	Signing configuration and claim interpretation must remain consistent
Application-level tenant authorization	Tenant permission depends on memberships and organization ancestry	Every tenant-owned query must include validated tenant context
Append-only point ledger	Preserves history and supports reconciliation	Point balances require aggregation and appropriate indexes
PostgreSQL booking constraints	Application checks cannot prevent every concurrency race	Migrations contain exclusion and partial-unique indexes
Independently runnable services	Maintains clear technical ownership and deployability	Each service needs its own Dockerfile, health endpoint and configuration
Provider-neutral integrations	Reduces dependency on a particular hosting vendor	Provider details belong in configuration and adapters
Persistent notification record	Users must retain notification history during delivery failures	WebSocket or Redis failure cannot remove or roll back the notification

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

4. Use-Case View

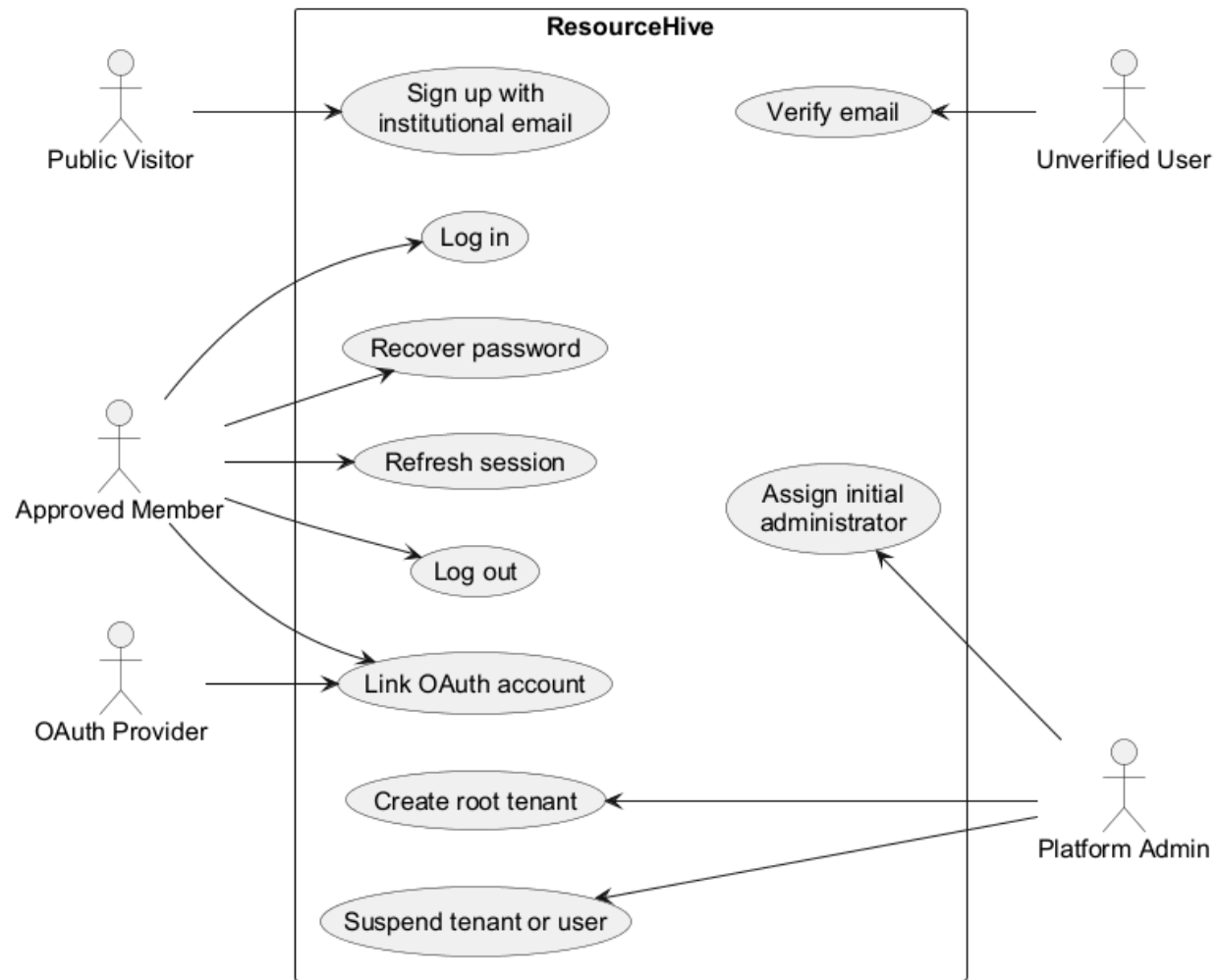


Figure 3. Identity and Tenant Onboarding Use Cases

Figure 3 summarizes identity and tenant-onboarding use cases. Visitors register, registered users verify their email, log in, or reset their password, and authenticated members refresh or end their session. Platform administrators create root tenants, assign initial administrators, and suspend tenants or users. OAuth account linking is shown as an optional approved integration.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

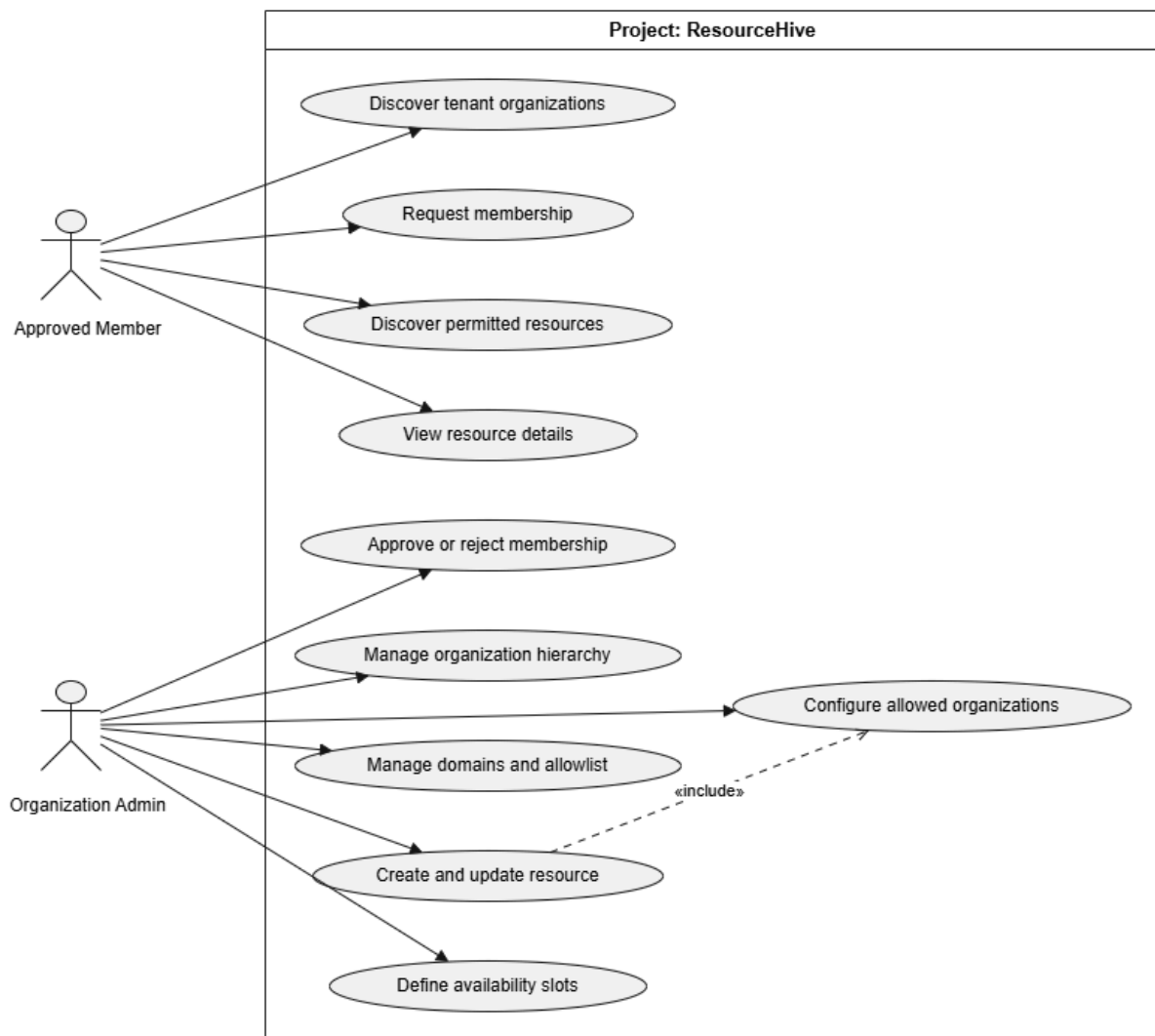


Figure 4. Organization, Membership, and Resource Use Cases

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

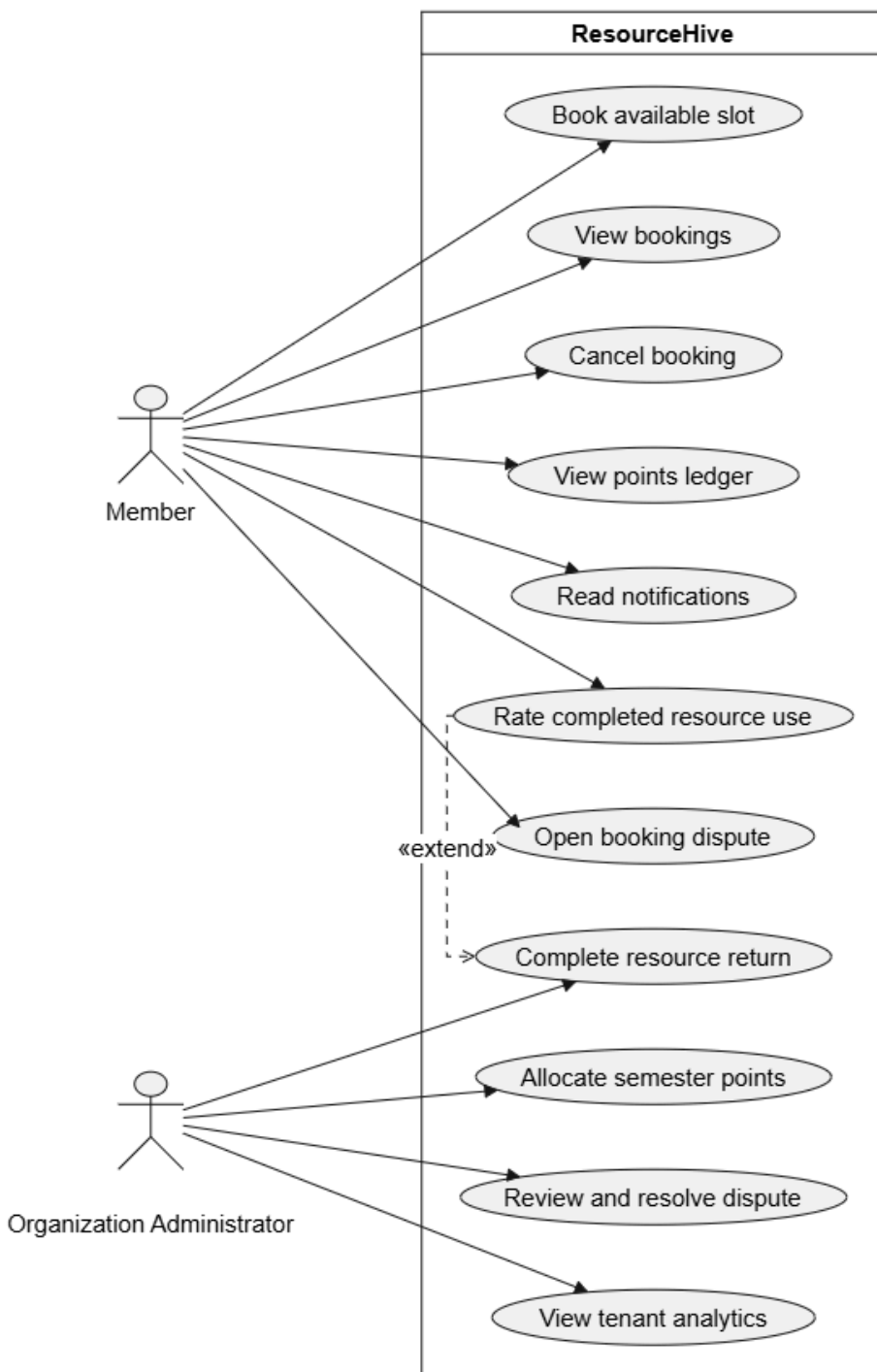


Figure 5. Booking, Points, Notifications, Ratings, and Disputes Use Cases

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

4.1 Use-Case Realizations

4.1.1 Sign up

Use case name	Sign up
Actor	Public Visitor
Description	A public visitor registers for a new account by providing their details and an institutional email address to gain access to the platform.
preconditions	The user must possess a valid institutional email address that is not already registered in the system.
Main flow	1. The user navigates to the platform's registration page and enter their name, email and password 2. The system validates the email domain against the configured allowlists of active root tenants and create inactive user record in database 3. The system generates a secure, time-limited verification token and dispatch email containing the verification link.
Successful end/post condition	An inactive user account is created, and a verification email is successfully sent to the user.
Fail end/post condition	Account creation is rejected if the email domain is invalid, unassociated with any root tenant, or if the email is already registered.
Extensions	If the user clicks the verification link within the allowed timeframe, the system activates the account.

4.1.2 Verify email

Use case name	Verify email
Actor	Public Visitor
Description	A user verifies their institutional email address using a secure token sent to them, confirming their identity.
preconditions	The user has successfully completed the "Sign up" process.
Main flow	1. user opens their email client and clicks the verification link 2. The system intercepts the request and extracts the token from the URL.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

	- The system validates that the token exists in the database, belongs to the requesting user, and has not expired. - The system updates the user's account status to "active"
Successful end/post condition	The user account becomes active, allowing the user to log into the system
Fail end/post condition	If the token is expired, invalid, or already used, the system displays an error message, and the account remains unverified.
Extensions	

4.1.3 Log In

Use case name	Log In
Actor	Approved Member, Organization Admin, Platform Admin
Description	An active user authenticates into the system using their credentials to securely access
preconditions	The user must have an active, verified account
Main flow	- The user navigates to the login portal and submits their email address and password. - The system checks whether the provided credentials match a registered user. - The system generates JWT token containing the user's ID, roles, and root tenant context.
Successful end/post condition	The user is successfully authenticated, receives an active session token, and is redirected to their respective dashboard.
Fail end/post condition	If the credentials do not match or the account is suspended, authentication is denied, and show error message
Extensions	<<N/A>>

4.1.4 Discover Organizations & Request Membership

Use case name	Discover Organizations & Request Membership
Actor	Approved Member
Description	A member browses available child organizations within their institution and submits a formal request to join.
preconditions	The user is authenticated and is verified as belonging to a specific root tenant.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

Main flow	1. The user navigates to the organization discovery dashboard. 2. The system retrieves and displays a list of active child organizations belonging strictly to the user's root tenant to enforce data isolation. 3. The user selects a specific child organization and clicks the "Request to Join" button. 4. The system dispatches a notification to the respective Organization Admin indicating a new pending request.
Successful end/post condition	A pending membership request is successfully recorded in the database, and the administrator is notified to take action.
Fail end/post condition	The request fails and is blocked if the user already has a pending request, or attempts to bypass tenant boundaries.
Extensions	<<N/A>>

4.1.5 Find Resources

Use case name	Find Resources
Actor	Approved Member
Description	A member searches for and views the details of available resources that their approved organizations have explicit permission to access.
preconditions	The user is authenticated and holds an 'Approved' membership status in at least one child organization.
Main flow	1. The user navigates to the resource directory. 2. The system identifies all organizations where the user currently has an 'Approved' status. 3. The system queries the database for active resources whose allowlists include any of the user's approved organizations. 4. The system displays the filtered, authorized list of resources to the user. 5. The user selects a specific resource to view its details, required point cost, and future availability slots.
Successful end/post condition	The user successfully views a list of resources and their details

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

Fail end/post condition	If the system cannot retrieve the data due to a backend service failure, an appropriate error state is displayed.
Extensions	The user can apply filters (date range, specific organization, or resource type) to narrow down the displayed list of resources.

4.1.6 Book a Resource Slot

Use case name	Book a Resource Slot
Actor	Approved Member
Description	A member reserves a specific time slot for an available resource, resulting in an atomic deduction of points from their account.
preconditions	The member must be authenticated, belong to an organization that is permitted to access the resource, and possess a point balance greater than or equal to the resource's cost. The target time slot must be available.
Main flow	1. The member browses the available resources and selects a specific item then selects an available time slot and submits a booking request. 2. The system applies a database lock on the selected slot to prevent concurrent double-booking. 3. The system records a negative point transaction to deduct the required cost from the member. 4. The system updates the slot status to "CONFIRMED" and assigns it to the member. 5. The system dispatches a booking confirmation notification to the member.
Successful end/post condition	The slot is successfully reserved, the exact point cost is deducted from the user's ledger, and a notification is sent.
Fail end/post condition	The booking is rejected if the slot is simultaneously claimed by another concurrent request, or if the user has insufficient points.
Extensions	<<N/A>>

4.1.7 Cancel Own Booking

Use case name	Cancel Own Booking
Actor	Approved Member

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

Description	A member cancels an upcoming resource booking they previously made, freeing the time slot and triggering an automatic refund of the deducted points.
preconditions	The user is authenticated and has an active, confirmed booking for a future time slot that has not yet passed the organization's cancellation cutoff time.
Main flow	1. The user navigates to their personal bookings management page. 2. The user selects a confirmed future booking and executes the "Cancel" action. 3. The system verifies that the current timestamp is before the designated cancellation cutoff time for that specific resource. 4. The system updates the booking status to 'Cancelled', making the time slot available for other members and records a positive transaction in the append-only ledger, refunding the exact point cost to the user's account. 5. The system notifies the user of the successful cancellation and updated point balance.
Successful end/post condition	The booking is officially canceled, the time slot becomes publicly available again, and the points are accurately refunded.
Fail end/post condition	Cancellation is rejected if the cutoff time has already passed.
Extensions	<<N/A>>

4.1.8 View Point History

Use case name	View Point History
Actor	Approved Member
Description	A member views their personal append-only ledger detailing all point transactions, including joining bonuses, booking deductions, and cancellation refunds.
preconditions	The user is authenticated.
Main flow	1. The user navigates to the point management dashboard.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

	- The system shows all append-only ledger transactions with details of each transaction (date, amount,description). - The system calculates the current total point balance and shows balance.
Successful end/post condition	The user successfully views their accurate, up-to-date point transaction history and total balance.
Fail end/post condition	If the ledger service is unreachable, an error message is displayed and the history fails to load.
Extensions	The user can filter the ledger history by specific date ranges or transaction types.

4.1.9 View and Manage Notifications

Use case name	View and Manage Notifications
Actor	Approved Member
Description	A member views system alerts, such as booking confirmations, membership approvals, or upcoming reservation reminders, and marks them as read.
preconditions	The user is authenticated.
Main flow	- The user clicks on the notifications icon in the application header. - The system retrieves the list of recent notifications for the user, sorting unread items at the top. - The user clicks an action to mark the notification as "Read".
Successful end/post condition	The notifications are successfully retrieved, viewed, and their state is updated to 'Read'.
Fail end/post condition	If the notification service is down, the alert dropdown will not load.
Extensions	The user can select a "Mark all as read" option to update the state of all unread notifications simultaneously.

4.1.10 Create and Manage Root Tenants

Use case name	Create and Manage Root Tenants
Actor	Platform Admin
Description	The platform administrator provides a new root tenant (university) to allow their respective members to utilize the platform in an isolated environment.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

preconditions	The actor must be authenticated and hold Platform Admin privileges.
Main flow	1. The Platform Admin navigates to the tenant management dashboard and selects "Create New Tenant". 2. The Admin inputs the tenant details, including the organization name and the permitted email registration domains. 3. The system validates that the provided domains are unique and not currently in use by another tenant. 4. The system creates the root organization record. 5. The Admin assigns initial administrative accounts to manage the new root tenant.
Successful end/post condition	A newly isolated root tenant is active in the system, and public visitors with matching domains can begin registering.
Fail end/post condition	Creation is aborted, and an error is returned if the requested email domains conflict with an existing root tenant.
Extensions	<<N/A>>

4.1.11 Create Child Organization

Use case name	Create Child Organization
Actor	Organization Admin
Description	An administrator creates a new nested organization (a specific club or department) within their overarching root tenant structure
preconditions	The admin must be authenticated and possess Organization Admin privileges within the root tenant.
Main flow	1. The Admin navigates to the organization management dashboard and selects "Create Child Organization". 2. The Admin inputs the required details (name, description, category,...) 3. The system validates the inputs and creates the child organization record in the database. 4. The system automatically assigns the creator as an Organization Admin for the newly created child organization.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

Successful end/post condition	The child organization is successfully created, stored in the database, and becomes visible for members to discover and request to join.
Fail end/post condition	Creation fails if a duplicate name is used or if the database connection is interrupted.
Extensions	<<N/A>>

4.1.12 Approve/Reject Membership

Use case name	Approve/Reject Membership
Actor	Organization Admin
Description	An organization administrator reviews a pending request from a member attempting to join their specific child organization.
preconditions	The admin must be authenticated, hold Organization Admin privileges for the target organization, and there must be at least one pending membership request.
Main flow	1. The Admin navigates to the organization's pending memberships dashboard. 2. The Admin reviews a specific user's request details and selects the "Approve" action. 3. The system updates the user's membership status and automatically credits the user with the organization's configured joining bonus points via a new ledger transaction. 4. The system dispatches an approval notification to the user.
Successful end/post condition	The user is added as an active member, receives the joining bonus points, and gains access to resources restricted to that organization.
Fail end/post condition	The approval fails if the requesting user's account has been deactivated or if the admin's privileges have been revoked during the process.
Extensions	If the Admin selects 'Reject', the system updates the membership status to 'Rejected' and notifies the user.

4.1.13 Create and Manage Resources

Use case name	Create and Manage Resources
Actor	Organization Admin

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

Description	An administrator creates a new shared resource (equipment, a study room,...) and explicitly defines which internal child organizations are permitted to book it.
preconditions	The admin must be authenticated and hold privileges to create resources within their organization.
Main flow	1. The Admin selects the 'Create Resource' option within their dashboard. 2. The Admin inputs the resource details, including name, description, and the point cost per booking. 3. The Admin selects one or more internal child organizations to add to the resource's allowlist. 4. The system validates that all selected organizations belong to the exact same root tenant to enforce tenant isolation. 5. The system saves the resource and generates its initial availability calendar.
Successful end/post condition	The resource becomes active and is immediately discoverable by approved members of the explicitly allowed organizations.
Fail end/post condition	Resource creation is blocked if the admin attempts to grant access to an organization outside of their root tenant boundary.
Extensions	The Admin may later edit the resource to modify its point cost, update the list of allowed organizations, or adjust availability time slots.

4.1.14 Manage Email Allowlist

Use case name	Manage Email Allowlist
Actor	Organization Admin
Description	An administrator adds specific email addresses to a pre-approved list, allowing those users to automatically bypass the manual membership approval process.
preconditions	The admin must be authenticated and have administrative rights for the target child organization.
Main flow	1. The Admin navigates to the organization's settings and selects the "Email Allowlist" section.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

	- The Admin enters a list of specific user email addresses. - The system validates provided emails and updates the organization's allowlist database record.
Successful end/post condition	The allowlist is updated. Any future membership request from a user with a matching email address will be automatically approved by the system.
Fail end/post condition	The update is rejected if improperly formatted email addresses are submitted.
Extensions	The Admin can also remove previously added email addresses from the allowlist to revoke auto-join privileges.

4.1.15 Configure Organization Rules

Use case name	Configure Organization Rules
Actor	Organization Admin
Description	An administrator customizes specific platform behaviors for their organization, such as setting the joining bonus points awarded to new members.
preconditions	The admin must be authenticated and hold Organization Admin privileges.
Main flow	- The Admin navigates to the "Organization Rules & Settings" dashboard. - The Admin modifies the "Joining Bonus" value to a specified integer. - The Admin adjusts any other available parameters such as resource cancellation cutoff limits... and submits the changes. - The system validates the inputs and updates the organization's configuration record in the database.
Successful end/post condition	The new rules are saved and immediately applied to all subsequent platform actions
Fail end/post condition	The system rejects the update if invalid data types are submitted.
Extensions	<<N/A>>

4.1.16 View Bookings

Use case name	View Bookings
Actor	Approved Member

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

Description	A member views their current and previous resource bookings and the status of each booking.
preconditions	The member must be authenticated and have an active account.
Main flow	1. The member navigates to the “My Bookings” page. 2. The system validates the member’s JWT locally. 3. The system retrieves only bookings belonging to the authenticated member. 4. The system displays each booking with its resource, slot, status, point cost, and relevant timestamps. 5. The member selects a booking to view its complete details.
Successful end/post condition	The member successfully views their tenant-scoped booking history and booking details.
Fail end/post condition	If the booking information cannot be retrieved, the system displays an appropriate error message.
Extensions	The user can filter bookings by status, resource, or date range.

4.1.17 Complete Resource Return

Use case name	Complete Resource Return
Actor	Organization Administrator
Description	An authorized organization administrator confirms that a resource has been returned or that its reserved use has been completed.
preconditions	The administrator must be authenticated and authorized to manage the organization that owns the resource. The booking must have a CONFIRMED status.
Main flow	1. The administrator navigates to the booking or return-management page. 2. The system displays confirmed bookings for resources managed by the administrator. 3. The administrator selects a booking and confirms the resource return. 4. The system validates the administrator’s authority and the current booking status.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

	5. The system changes the booking status to COMPLETED and records the completion timestamp. 6. The system sends a completion notification to the member.
Successful end/post condition	The booking is marked as completed, the completion timestamp is recorded, and the member is notified.
Fail end/post condition	Completion is rejected if the administrator is unauthorized or the booking has already been cancelled or completed.
Extensions	After completion, the member may rate the resource or open an eligible dispute.

4.1.18 Allocate Semester Points

Use case name	Allocate Semester Points
Actor	Organization Administrator
Description	An organization administrator allocates points to eligible members for a particular academic semester.
preconditions	The administrator must be authenticated and have administrative authority over the target organization. The semester must be registered under the same root tenant.
Main flow	1. The administrator navigates to the semester-points management page. 2. The administrator selects the relevant semester and organization. 3. The administrator selects the eligible members and enters the number of points to allocate. 4. The system verifies that the semester, organization, and selected members belong to the same root tenant. 5. The system records a positive SEMESTER_ALLOCATION transaction for each eligible member. 6. The system recalculates and displays the members' updated point balances.
Successful end/post condition	The semester points are allocated to the selected members and recorded in the append-only points ledger.
Fail end/post condition	The allocation is rejected if the administrator is unauthorized, a member is ineligible, the allocation crosses a tenant boundary, or the same allocation has already been made.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

Extensions	The administrator may allocate points to multiple eligible members in a single operation.
-------------------	---

4.1.19 Rate Completed Resource Use

Use case name	Rate Completed Resource Use
Actor	Approved Member
Description	A member submits a rating and an optional comment for a resource after completing their booking.
preconditions	The user must be authenticated and must own a COMPLETED booking for the resource. The booking must not already have a rating.
Main flow	1. The member opens a completed booking. 2. The system displays the option to rate the resource. 3. The member selects a rating from one to five and optionally enters a comment. 4. The system validates booking ownership, completion status, tenant scope, resource matching, and rating range. 5. The system stores the resource rating. 6. The resource's average rating is recalculated from its eligible ratings.
Successful end/post condition	The rating is stored and becomes part of the resource's rating information
Fail end/post condition	The rating is rejected if the score is invalid, the booking is incomplete, the booking belongs to another user, or a rating already exists for it.
Extensions	The user may include an optional written comment with the rating.

4.1.20 Open Booking Dispute

Use case name	Open Booking Dispute
Actor	Approved Member
Description	A member opens a dispute concerning one of their eligible resource bookings.
preconditions	The user must be authenticated, own the relevant booking, and have no existing dispute for that booking.
Main flow	1. The member opens the relevant booking. 2. The member selects "Open Dispute."

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

	- The member provides a reason, description, and optional supporting evidence. - The system validates booking ownership, eligibility, tenant scope, and uniqueness. - The system creates a BookingDispute with the status OPEN. - The system creates an append-only OPENED dispute event identifying the member as the actor. - The system notifies the authorized organization administrators.
Successful end/post condition	A tenant-scoped dispute is created in the OPEN state, and the administrator is notified.
Fail end/post condition	The dispute is rejected if the booking is unauthorized, ineligible, cross-tenant, or already has a dispute.
Extensions	The member may submit additional evidence while the dispute remains open or under review.

4.1.21 Review and Resolve Dispute

Use case name	Review and Resolve Dispute
Actor	Organization Administrator
Description	An authorized organization administrator reviews a booking dispute and resolves or rejects it.
preconditions	The administrator must be authenticated and have authority over the organization responsible for the disputed resource. The dispute must be OPEN or UNDER_REVIEW.
Main flow	- The administrator opens the dispute-management dashboard. - The system displays disputes within the administrator's tenant and organization authority. - The administrator selects an open dispute. - The system changes its status to UNDER_REVIEW and records an append-only event. - The administrator reviews the booking, reason, description, and submitted evidence. - The administrator enters resolution notes and selects RESOLVED or REJECTED.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

	7. The system records the reviewer, final status, resolution notes, and resolution timestamp. 8. The system adds the corresponding append-only dispute event and notifies the member.
Successful end/post condition	The dispute is marked as RESOLVED or REJECTED, its audit history is preserved, and the member is notified.
Fail end/post condition	The operation is rejected if the administrator lacks authority, the dispute belongs to another tenant, or required resolution information is missing.
Extensions	The administrator may request or add evidence and notes before reaching a decision. Any point correction must be recorded as a new append-only point transaction.

4.1.22 View Tenant Analytics

Use case name	View Tenant Analytics
Actor	Organization Administrator
Description	An authorized organization administrator views analytics for organizations and resources within their permitted tenant hierarchy.
preconditions	The administrator must be authenticated and possess an approved ADMIN membership for the target organization or an authorized ancestor.
Main flow	1. The administrator opens the analytics dashboard. 2. The system validates the administrator's tenant and organization authority. 3. The administrator selects an organization, date range, and optional filters. 4. The system derives analytics from tenant-scoped resources, bookings, cancellations, point transactions, ratings, disputes, and memberships. 5. The system displays summaries and utilization measurements within the administrator's authority.
Successful end/post condition	The administrator views accurate analytics limited to their authorized tenant and organization hierarchy.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

Fail end/post condition	Access is rejected if the administrator requests information from another tenant or an organization outside their administrative authority.
Extensions	The administrator may filter results by date, organization, resource, booking status, or activity type. A root-organization administrator may view tenant-wide analytics.

5. Logical View

5.1 Overview

The ResourceHive design model is decomposed into logically independent subsystems (microservices) to ensure high cohesion, loose coupling, and clear boundaries of ownership. The system logic is primarily divided across four backend domains. Identity, Resource, Booking, and Notification.

These subsystems are supported by shared internal packages to maintain architectural consistency without duplicating critical business logic. Tenant isolation is logically enforced across all packages by scoping queries and actions to a specific root organization id rather than relying on separate databases.

5.2 Architecturally Significant Design Packages

Below are the significant packages and their primary classes that govern the platform's core behavior.

5.2.1 Identity Package

This package is exclusively responsible for user authentication, registration, and credential management.

- User: Represents individuals interacting with the platform.
 - Attributes: id, email, password_hash, status (ACTIVE/SUSPENDED), platform_role.
 - Responsibilities: Acts as the global identity across all tenant organizations.
- VerificationToken / RefreshToken:
 - Responsibilities: Manages time-limited, cryptographically hashed tokens for email verification and session renewal.

5.2.2 Resource & Organization Package

This package manages the multi-tenant hierarchy and the shared facilities/items.

- Organization: Represents either a root tenant (e.g., a university) or a child unit (e.g., a club or faculty).
 - Attributes: id, parent_id, root_organization_id, status, joining_bonus.
 - Responsibilities: Enforces tenant boundaries. It cannot be its own parent, nor can the hierarchy contain cycles.
- Membership: Connects a User to an Organization.
 - Attributes: role (ADMIN/MEMBER), status (PENDING/APPROVED/REJECTED).
- Resource: Represents a bookable entity.
 - Attributes: id, point_cost, cancellation_period, root_organization_id.
 - Responsibilities: Defines what can be booked and explicitly which organizations (and their approved members) are allowed to access it.

5.2.3 Booking & Ledger Package

This package handles availability, reservations, append-only point transactions, and post-booking actions like ratings and disputes.

- Booking: Represents a user's reservation of a resource slot.
 - Attributes: id, status (CONFIRMED/CANCELLED/COMPLETED), completedAt, cancelledAt.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

- Responsibilities: Ensures only one active booking exists per slot and correctly tracks lifecycle timestamps.
- PointTransaction: Represents an append-only entry in the points ledger.
 - Attributes: type (JOIN_BONUS, BOOKING, BOOKING_REFUND, SEMESTER_ALLOCATION), amount.
 - Responsibilities: Maintains the integrity of a user's point balance without modifying or deleting past transactions.
- BookingDispute: Manages conflict resolution for completed or cancelled bookings.
 - Attributes: reason, status (OPEN/UNDER_REVIEW/RESOLVED), resolutionNotes.

5.2.4 Notification Package

This package manages the delivery and persistence of system alerts.

- Notification:
 - Attributes: id, type, title, message, readAt, createdAt.
 - Responsibilities: Persists recipient-scoped alerts and tracks read statuses to ensure users can view history even if real-time WebSockets fail.

5.3 Domain Class Diagrams

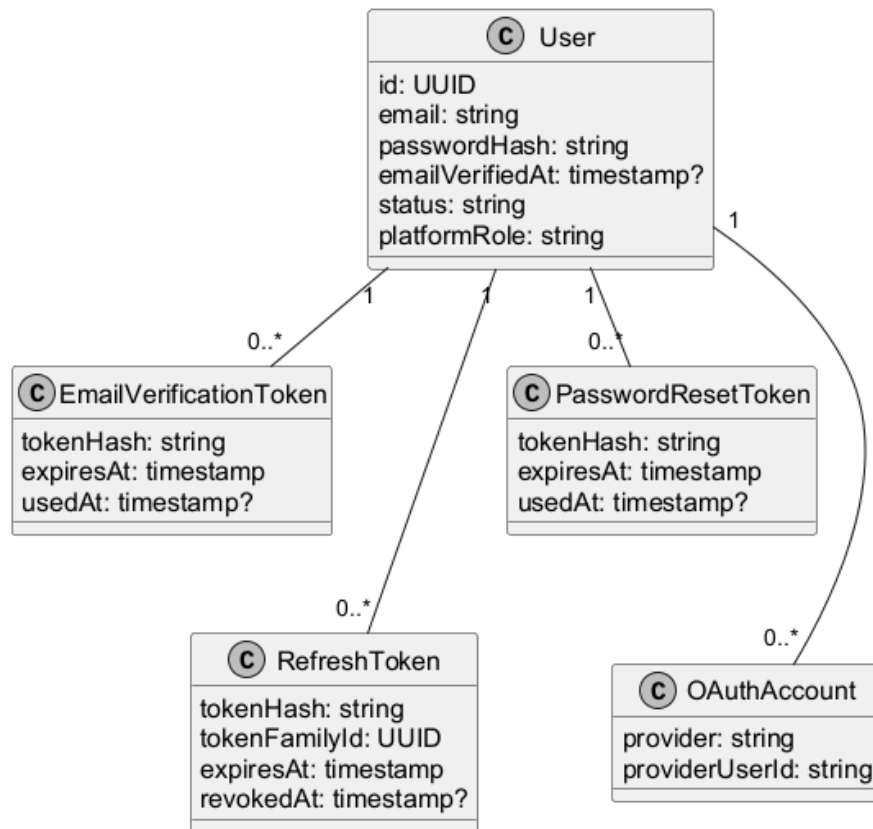


Figure 6(a). Identity and Authentication Domain Classes

Figure 6(a) shows the global User identity and its authentication records. Email verification and password reset tokens are time-limited and single-use. Refresh tokens represent renewable sessions, while OAuthAccount stores an optional approved provider link.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

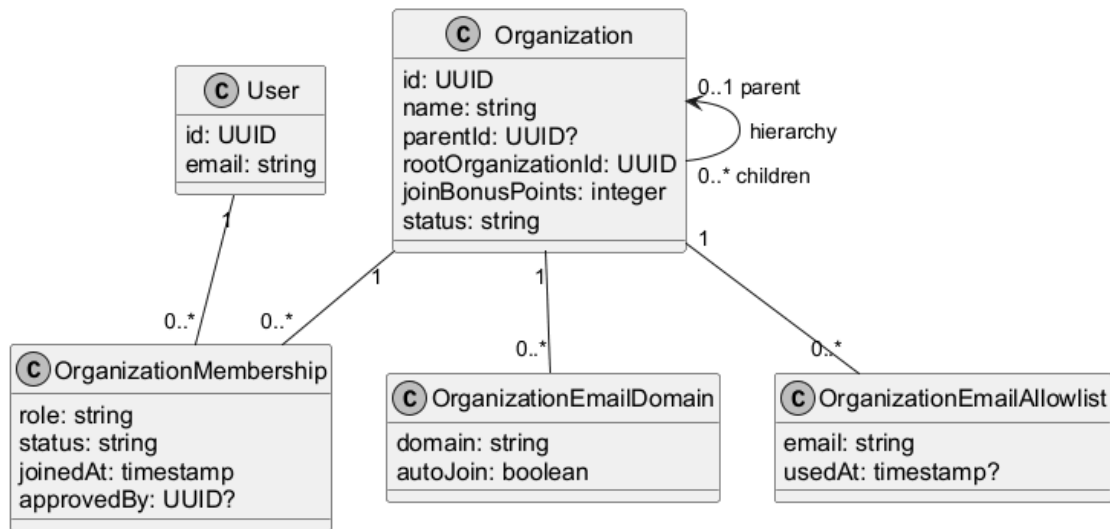


Figure 6(b). Tenant and Membership Domain Classes

Figure 6(b) shows how users join organizations through memberships. Organizations use parentId to form a hierarchy and rootOrganizationId to identify the tenant boundary. Email domains and allowlist records control registration and membership access, while joinBonusPoints stores the points granted by an organization.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

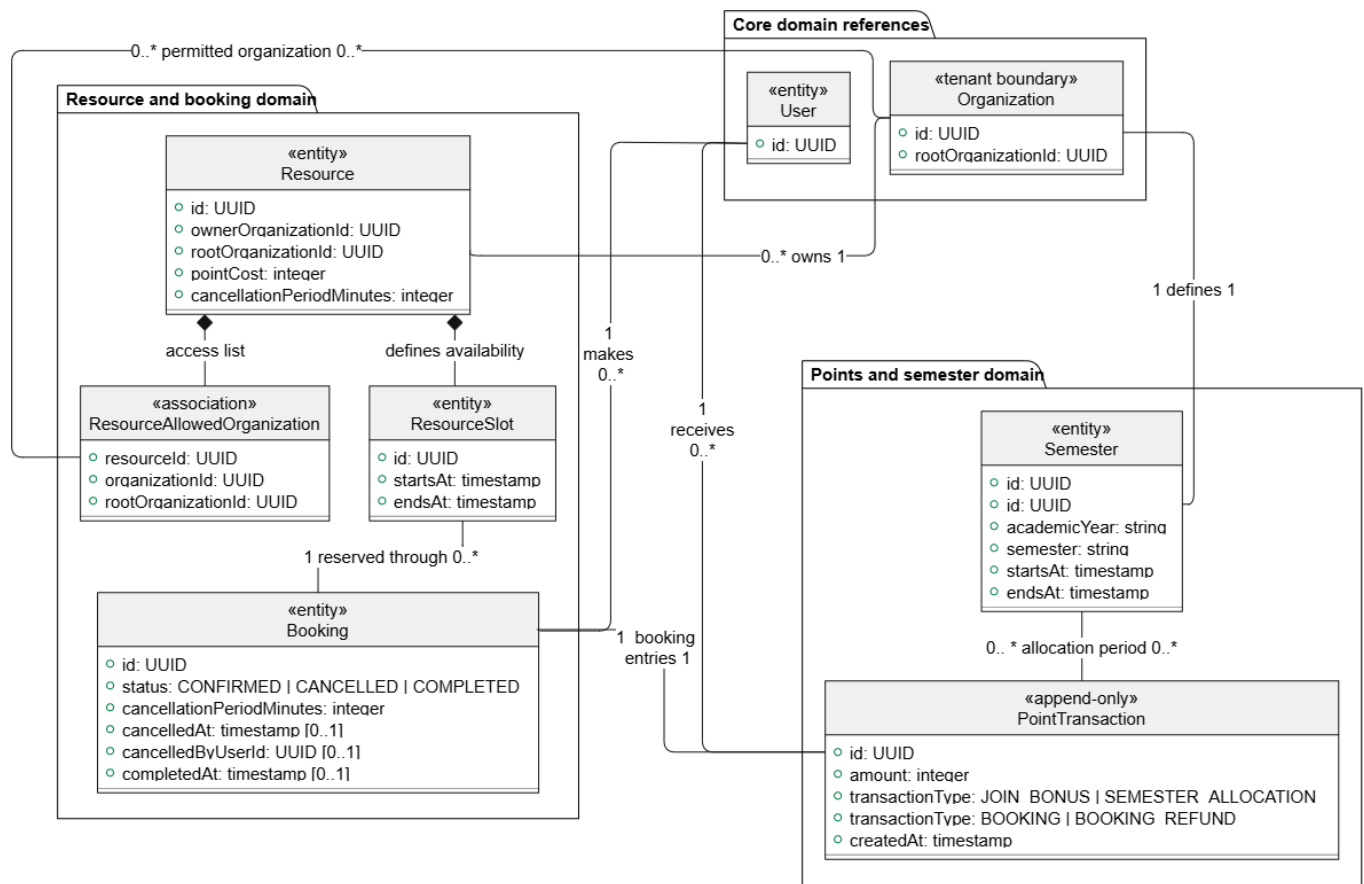


Figure 7. Resource, Booking, Points, and Semester Domain Classes

Figure 7 connects organization-owned resources to allowed organizations, availability slots, and user bookings. PointTransaction is the append-only points ledger used for booking deductions, refunds, joining bonuses, and semester allocations. Booking and point changes are performed together when a booking is created.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

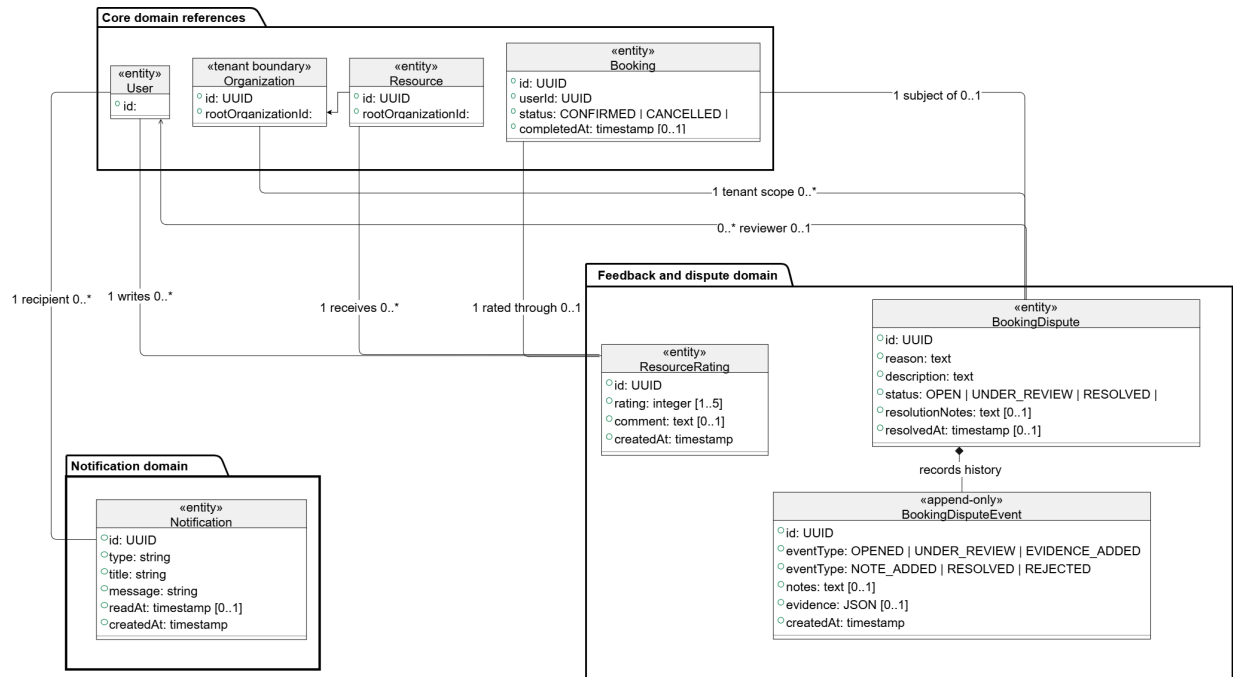


Figure 8. Rating, Dispute, and Notification Domain Classes

Figure 8, shows the domain classes supporting ratings, booking disputes, dispute-event history, and user notifications. A completed booking may receive one rating and may be associated with one dispute. Each dispute maintains an append-only history of status changes, evidence, and resolution actions. Notifications are stored for individual recipients and may reference relevant booking or dispute events.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

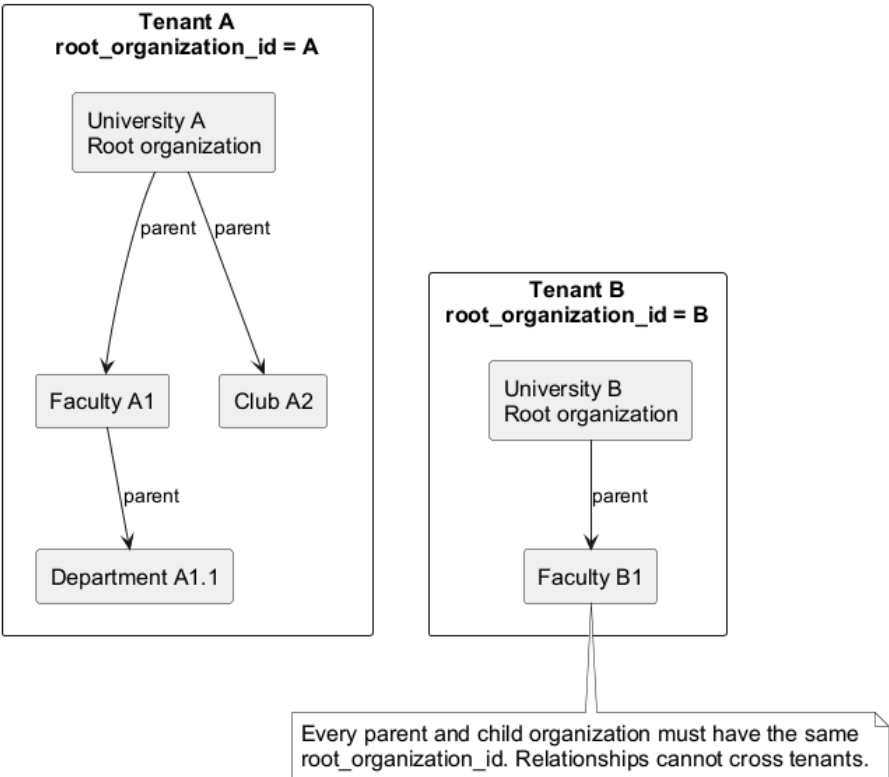


Figure 9(a). Root Tenant and Organization Boundaries

Figure 9(a) shows two independent root tenants and their child organizations. Every parent-child relationship must remain inside one `root_organization_id`. The database rejects organization relationships that cross a tenant boundary.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

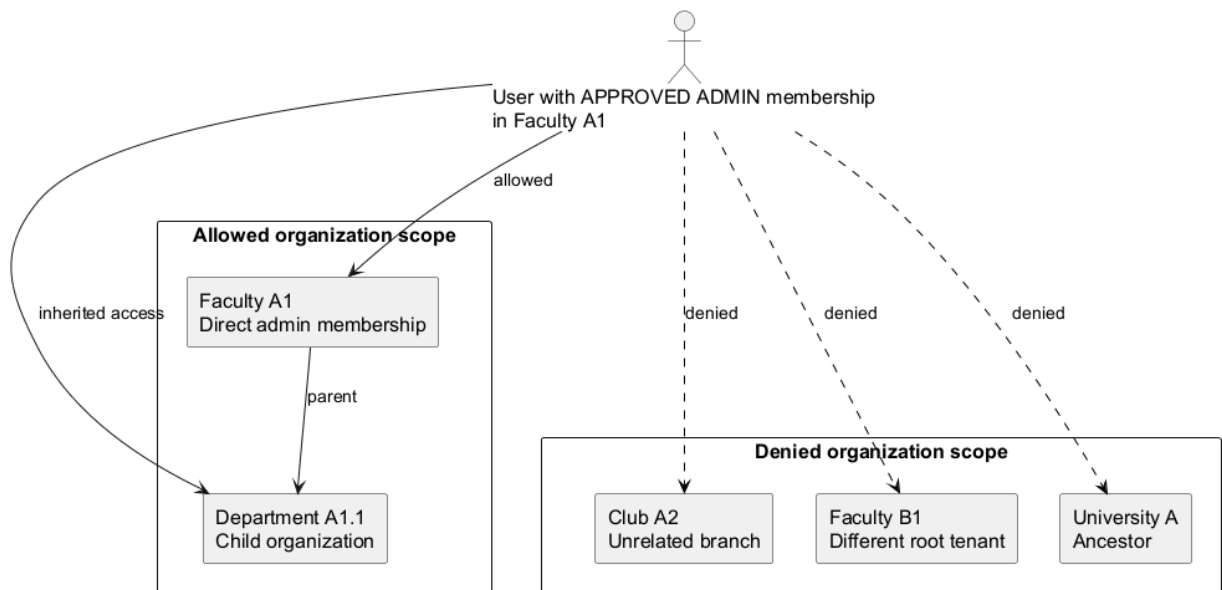


Figure 9(b). Inherited Organization Administration

Figure 9(b) shows the authorization example used by ResourceHive. A user with an approved ADMIN membership in Faculty A1 can manage Faculty A1 and its child Department A1.1. The user cannot manage an ancestor, an unrelated branch, or an organization in another root tenant.

The Class Diagram illustrates the structural relationships and strict tenant boundaries within the ResourceHive architecture:

1. **Tenant Boundary (Organization & Resource):** The diagram highlights that both Organization and Resource entities contain a `rootOrganizationId`. This structural relationship is critical for tenant isolation, ensuring that a resource and its owner always belong to the same root tenant.
2. **Booking Lifecycle (Booking):** The Booking class acts as a central junction, linking a User (via `userId`) to a specific resource slot. It manages the state machine of the reservation with its status attribute (CONFIRMED, CANCELLED) and tracks fulfillment via the `completedAt` timestamp.
3. **Feedback & Dispute Domain:**
 - The diagram shows that a Booking has a 1 to 0..1 relationship with ResourceRating, meaning a completed booking can optionally be rated exactly once.
 - Similarly, a Booking has a 1 to 0..1 relationship with BookingDispute. The BookingDispute class is composed of BookingDisputeEvent entities. The `<<append-only>>` stereotype on the event class dictates that all state changes, evidence additions, and resolutions are recorded as a permanent, immutable history.
4. **Recipient-Scoped Alerts (Notification):** The Notification class has a many-to-one dependency on the User class (as a recipient), utilizing `readAt` and `createdAt` timestamps to manage the user's inbox state.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

6. Process View

The Process View explains how ResourceHive components collaborate at runtime. Requests travel from the Next.js application through the Nginx API Gateway to the responsible NestJS service. Nginx only routes requests; the receiving service validates input, verifies the JWT locally, applies tenant and role authorization, and performs database operations through Prisma and PostgreSQL.

Correctness-sensitive operations use PostgreSQL transactions and constraints. Cross-service communication is limited to cases where another service owns the required responsibility, such as notification delivery.

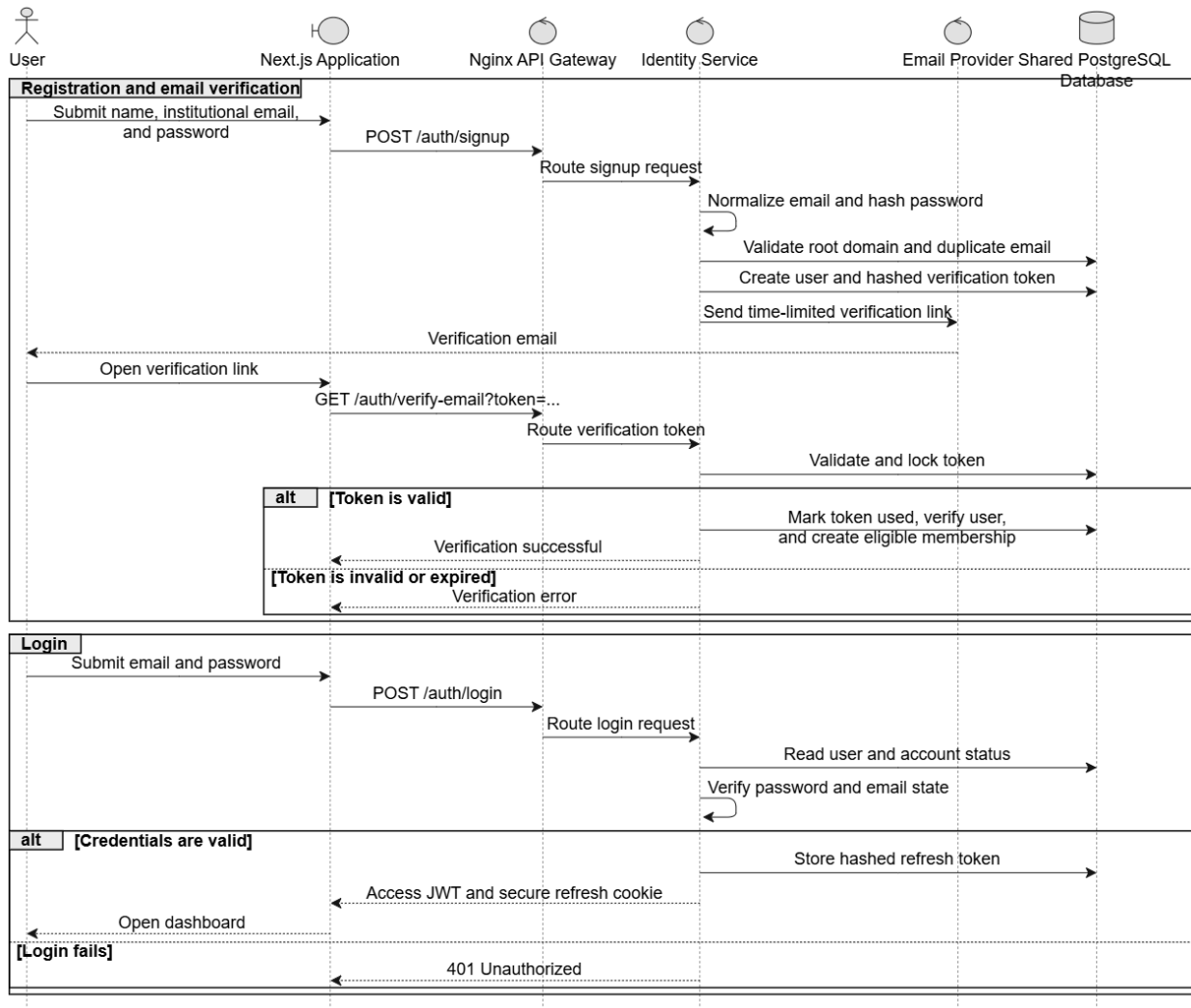


Figure 10. Identity Onboarding and Authentication

Figure 10, shows the registration, email-verification, and login processes. During registration, the Identity Service validates the institutional email, hashes the password, and stores the user and verification token before sending a time-limited verification link. Successful verification activates the user and creates any eligible organization membership. Existing verified users may log in to receive an access JWT and secure refresh token.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

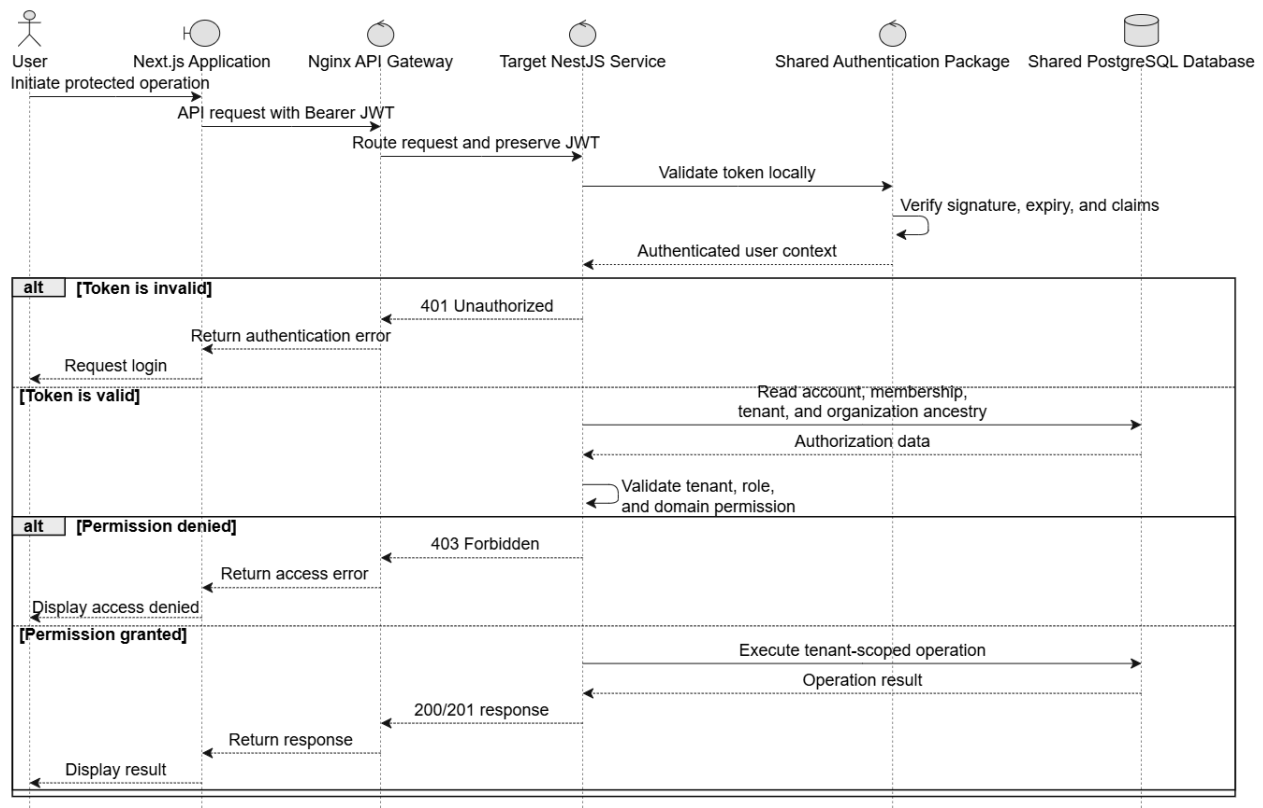
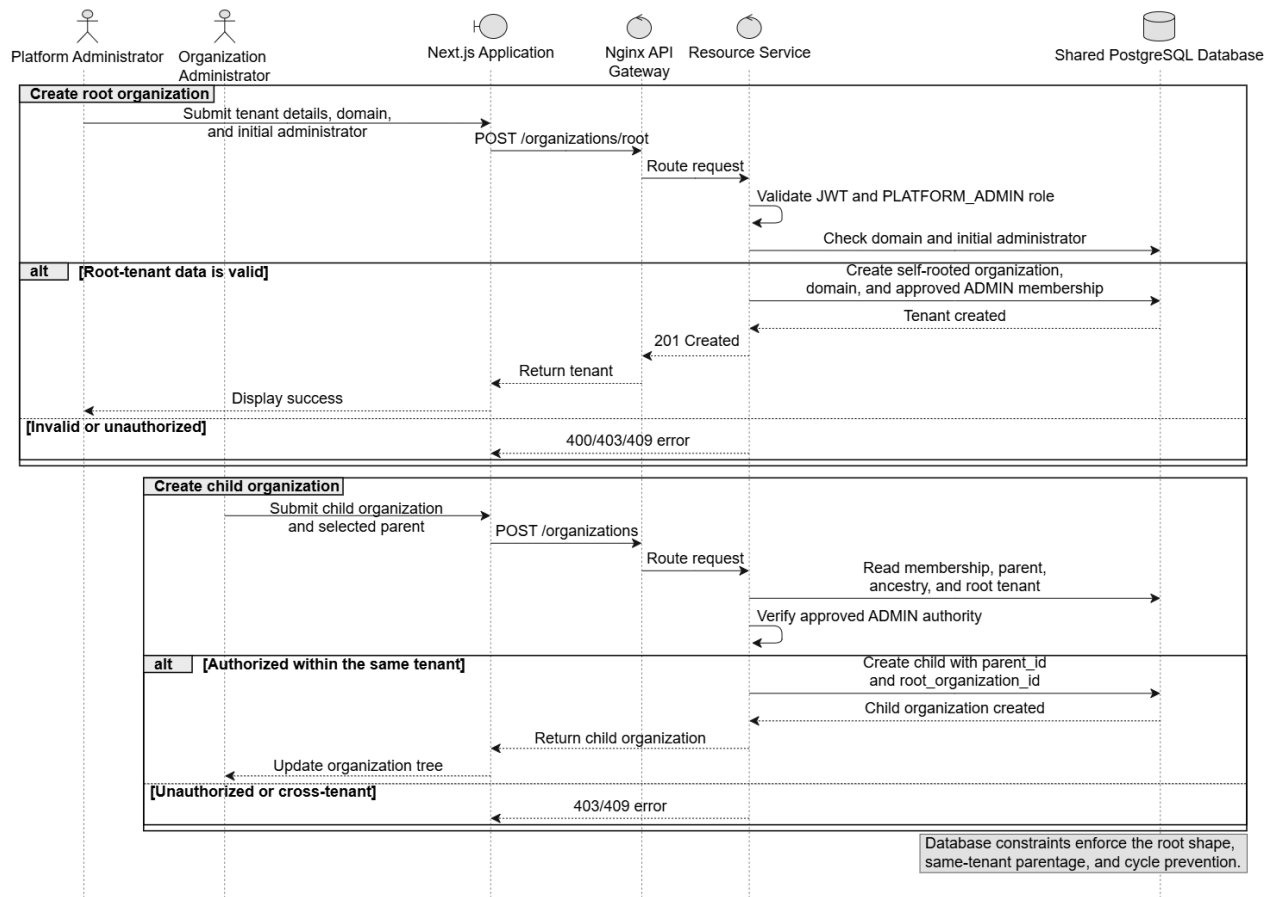


Figure 11. Authenticated Request and Role Validation

Figure 11, shows how ResourceHive processes a protected request. Nginx routes the request to the responsible backend service but does not authorize it. The target service validates the JWT locally and retrieves authoritative account, tenant, membership, role, and organization-hierarchy information. Unauthorized requests return 401 or 403, while authorized requests continue as tenant-scoped database operations.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	



12. Tenant and Organization Hierarchy Setup

Figure 12, shows the creation of root and child organizations. A Platform Administrator creates a root organization, configures its institutional domain, and assigns its first approved Organization Administrator. An authorized Organization Administrator may then create child organizations under an organization they manage. Database constraints preserve the root structure, tenant boundary, valid parent relationships, and cycle-free hierarchy.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

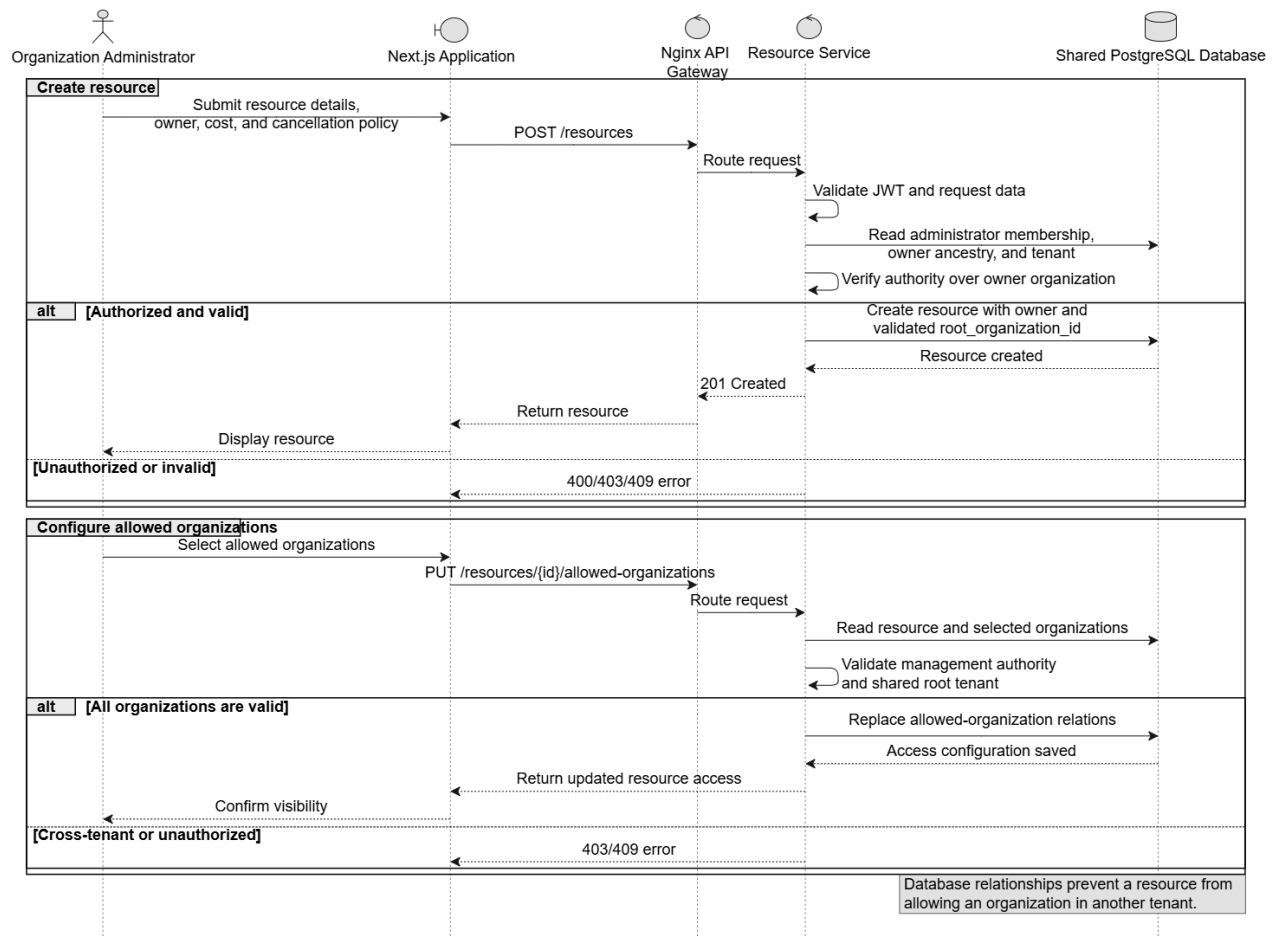


Figure 13. Resource Creation and Allowlist Configuration

Figure 13, shows how an Organization Administrator creates a resource and configures which organizations may access it. The Resource Service validates the administrator’s approved membership, organization authority, and tenant context before creating the resource. When the access list is updated, every selected organization must belong to the resource’s root tenant, preventing cross-tenant resource visibility.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

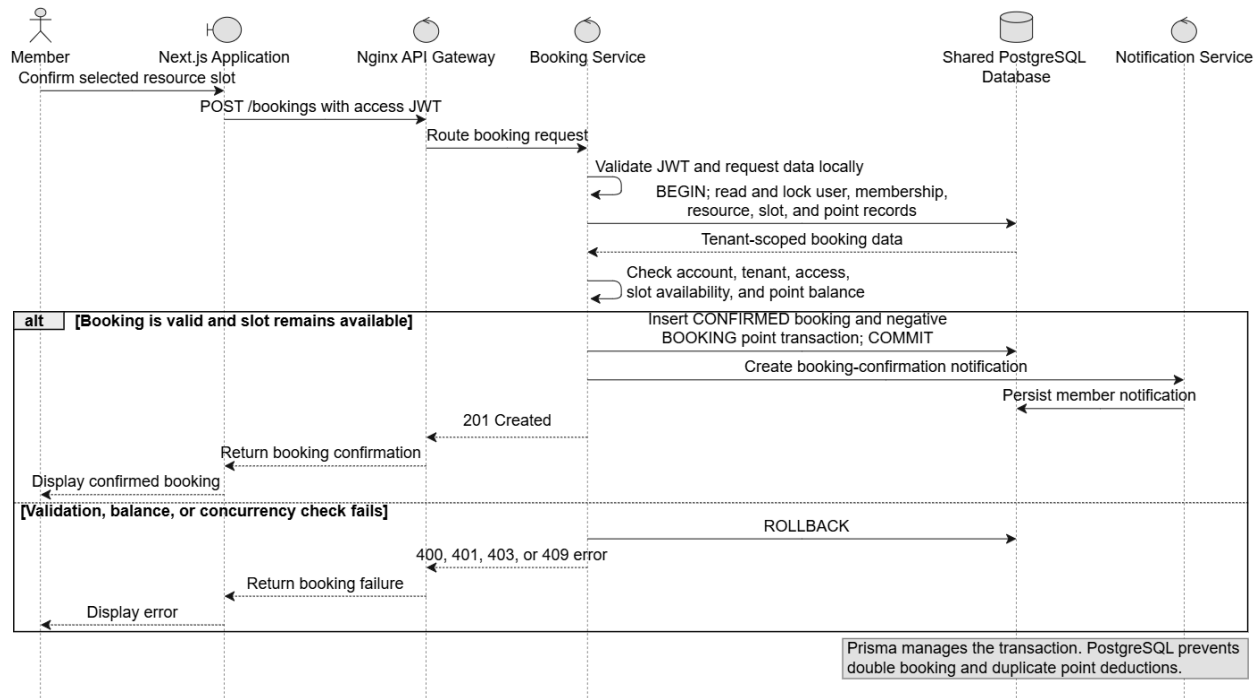


Figure 14. Atomic Booking and Point Deduction

Figure 14, shows how the Booking Service validates the account, tenant, membership, resource access, slot availability, and point balance. It then creates the CONFIRMED booking and negative BOOKING point transaction in one PostgreSQL transaction. Both records commit together, or both are rolled back. A confirmation notification is stored only after the booking succeeds.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

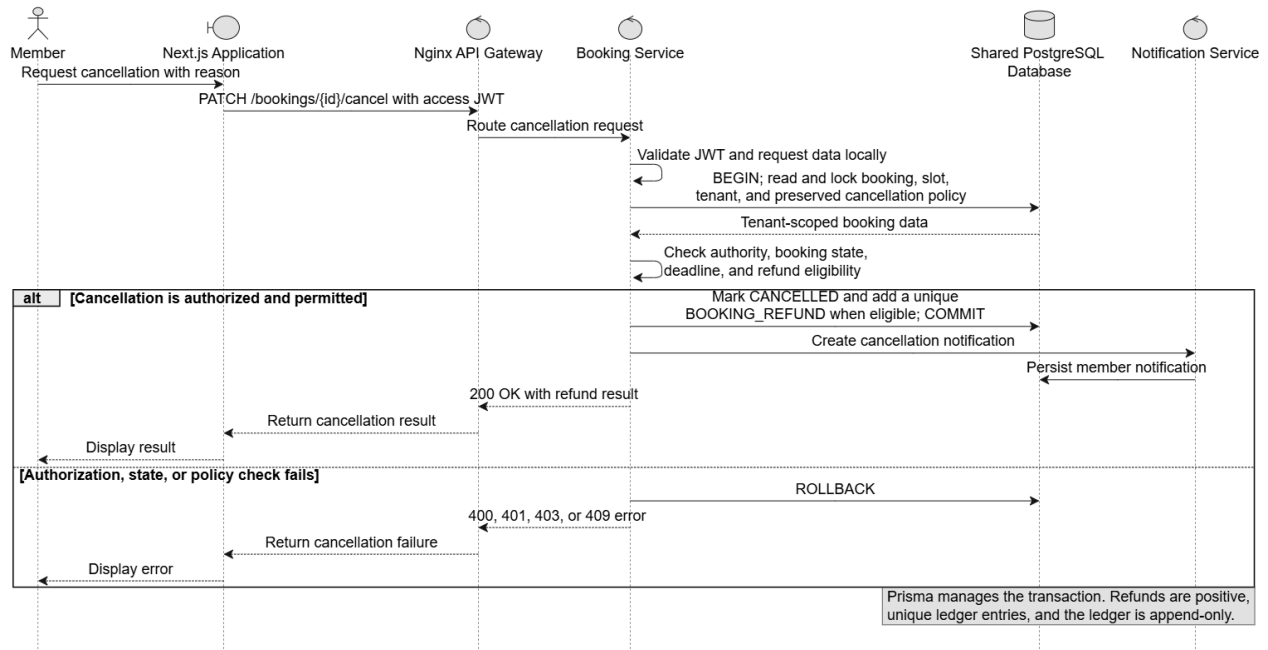


Figure 15. Booking Cancellation and Refund

Figure 15, shows how the Booking Service validates booking ownership or administrative authority, tenant scope, booking state, and the preserved cancellation policy. An approved cancellation changes the booking to CANCELLED and creates a positive BOOKING_REFUND transaction when eligible. The cancellation and refund are committed atomically before the member is notified.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

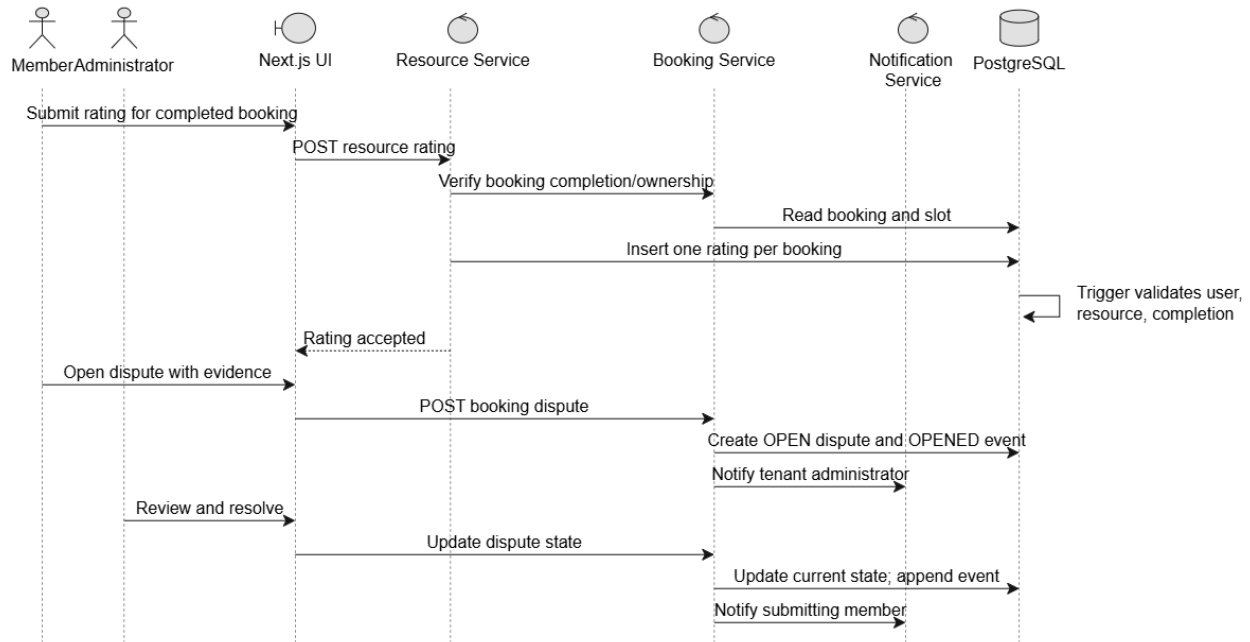


Figure 16. Rating and Dispute Processing

Figure 16, shows that a member may rate a resource only after completing their own booking. PostgreSQL validates the user, booking, resource, completion status, and one-rating-per-booking rule. It also shows how a member opens a dispute and how an authorized Organization Administrator reviews and resolves it. Each dispute action is recorded in append-only history, and the affected user is notified.

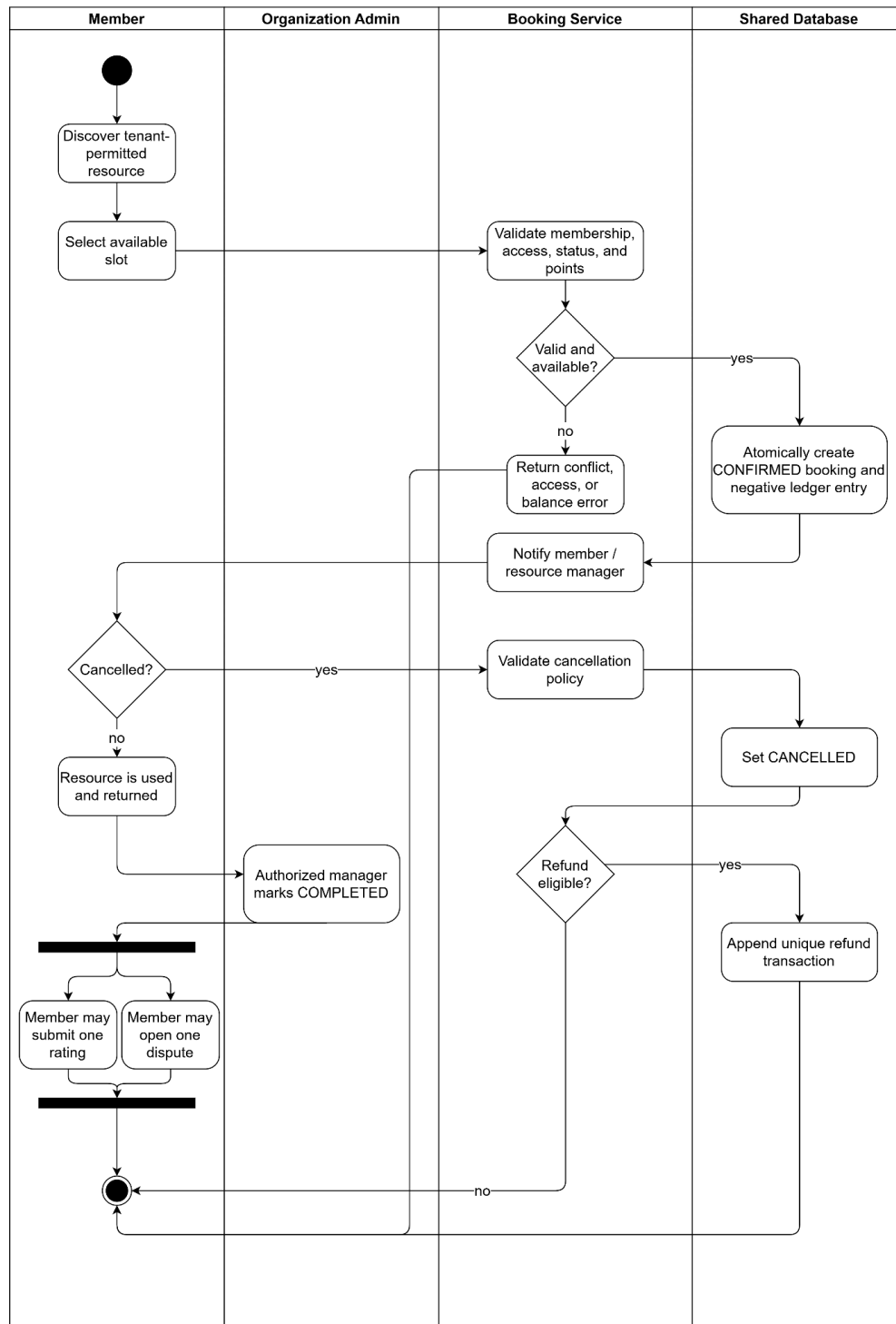


Figure 17. Booking Lifecycle Activity

Figure 17, summarizes the complete booking lifecycle. A member discovers an authorized resource, selects a slot, and undergoes access, availability, and point validation. A successful request creates a booking and deduction atomically. The booking may later be cancelled with an eligible refund or completed after the resource is used and returned. Completed bookings may support a rating and dispute.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

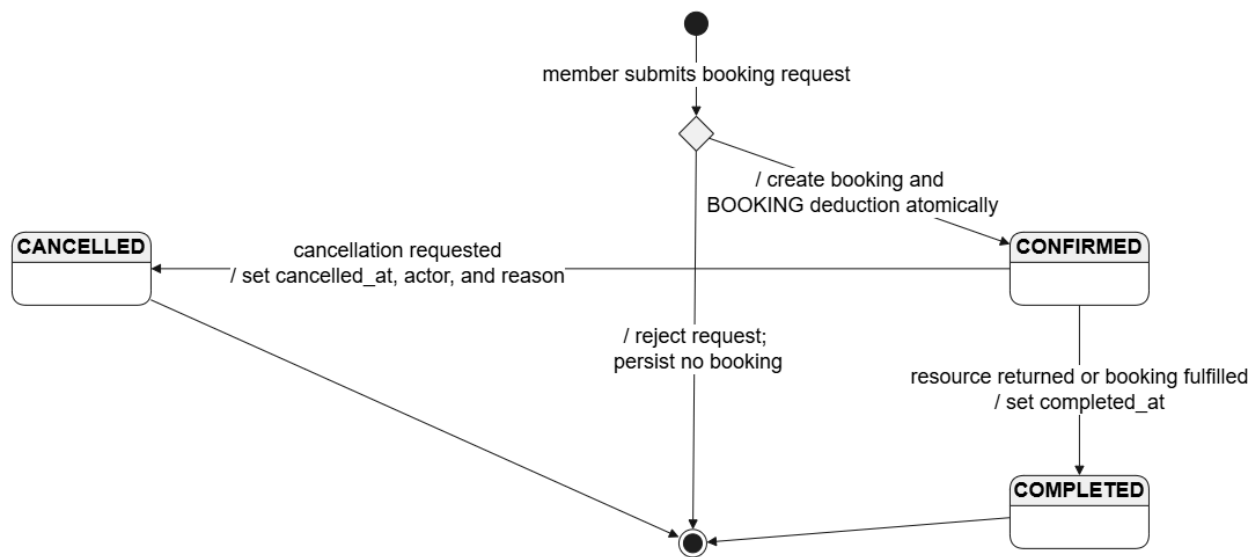


Figure 18. Booking State Model

Figure 18, shows the three persisted booking states: CONFIRMED, CANCELLED, and COMPLETED. A valid booking request creates a CONFIRMED booking. An authorized cancellation moves it to CANCELLED, while an authorized return or completion moves it to COMPLETED. Cancelled and completed bookings are terminal states. Ratings and disputes do not change a completed booking's state.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

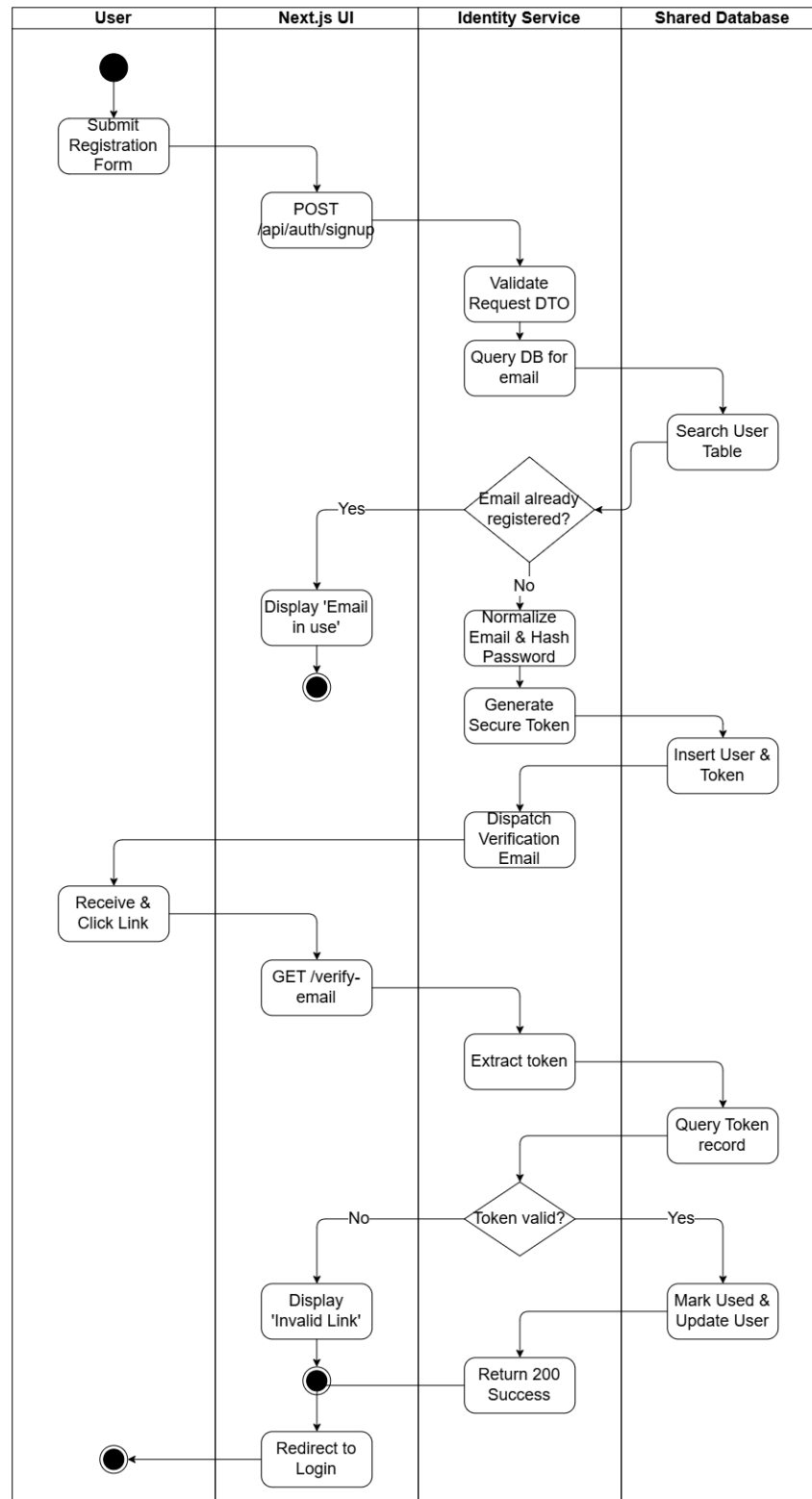


Figure 19. User Registration and Email Verification Activity

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

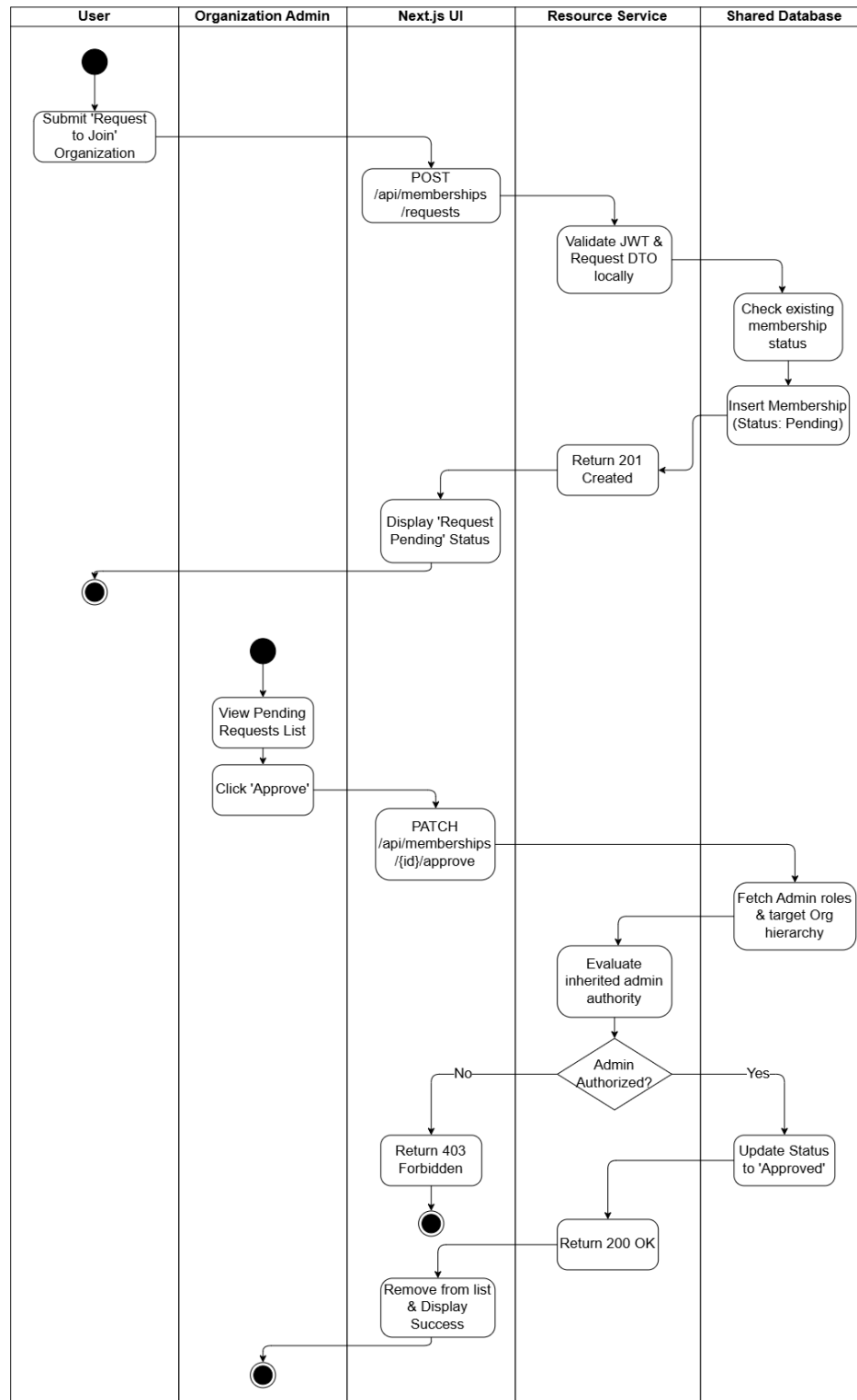


Figure 20. Membership Request and Admin Approval Activity

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

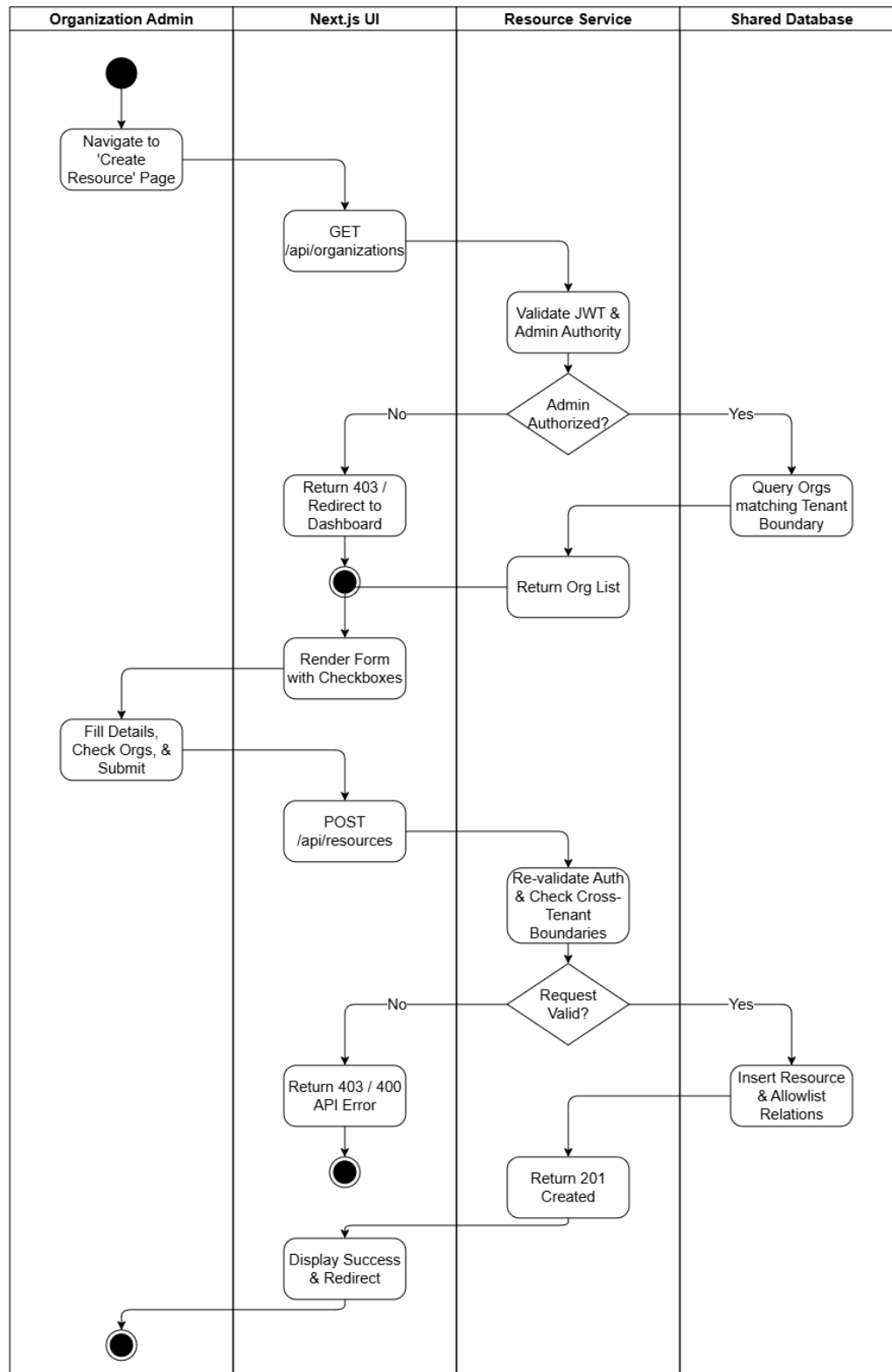


Figure21. Resource Creation and Visibility Configuration Activity

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

Figure 19 shows the registration and email-verification workflow. The user submits the registration form, after which the Identity Service validates the details, checks for an existing email address, hashes the password, and stores the user with a secure verification token. The user then opens the emailed verification link. A valid token is marked as used and verifies the user, while an invalid or expired token produces an error. After successful verification, the user is redirected to the login page.

Figure 20, shows how a user requests membership in an organization and how the request is reviewed. A valid request creates a membership with PENDING status. An Organization Administrator may approve it only after the Resource Service verifies the administrator's membership, authority, organization hierarchy, and tenant scope. An unauthorized approval attempt is rejected, while successful approval changes the membership to APPROVED.

Figure 21, shows how an Organization Administrator creates a resource and configures its organizational visibility. The Resource Service verifies the administrator's identity and management authority before accepting the resource details. It also confirms that the owner organization and selected allowed organizations belong to the same root tenant. A valid request creates the resource and its allowlist relationships, while invalid or cross-tenant requests are rejected.

6.1 Concurrency and Failure Handling

ResourceHive combines application validation with PostgreSQL integrity controls:

- A GiST exclusion constraint prevents overlapping slots for the same resource.
- A partial unique index permits only one CONFIRMED or COMPLETED booking per slot.
- Booking creation and point deduction occur in one transaction.
- Unique indexes prevent duplicate bonuses, allocations, deductions, and refunds.
- Append-only triggers prevent modification or deletion of point and dispute history.
- Transactional token use prevents one verification token from succeeding more than once.
- Notification, email, Redis, or WebSocket failure does not undo a committed domain operation.

The system returns controlled validation, authentication, authorization, balance, state, and conflict errors without exposing another tenant's protected information.

7. Deployment View

The Deployment View describes how ResourceHive applications and services are placed within the initial production environment. The Next.js application is hosted separately from the backend, while the backend services run as Docker containers on one Docker-capable host.

The browser accesses the platform through configured DNS and HTTPS endpoints. API requests are sent to the Nginx API Gateway, which routes them to the appropriate private service container. All backend services connect to one external PostgreSQL database using encrypted transport. The Identity and Notification Services communicate with an approved SMTP or email provider.

Below figure 22, shows the browser accessing the Next.js application through DNS and HTTPS. The frontend sends API requests to the Nginx container on the backend host. Nginx is the only publicly reachable backend component and routes requests to the private Identity, Resource, Booking, and Notification containers through the Docker network.

It also shows that all four backend services connect to the shared PostgreSQL provider using encrypted SQL connections. The Identity and Notification Services use an approved SMTP or email-provider interface. Specific providers such as Vercel, AWS, or Neon are deployment options rather than dependencies of the ResourceHive business logic.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

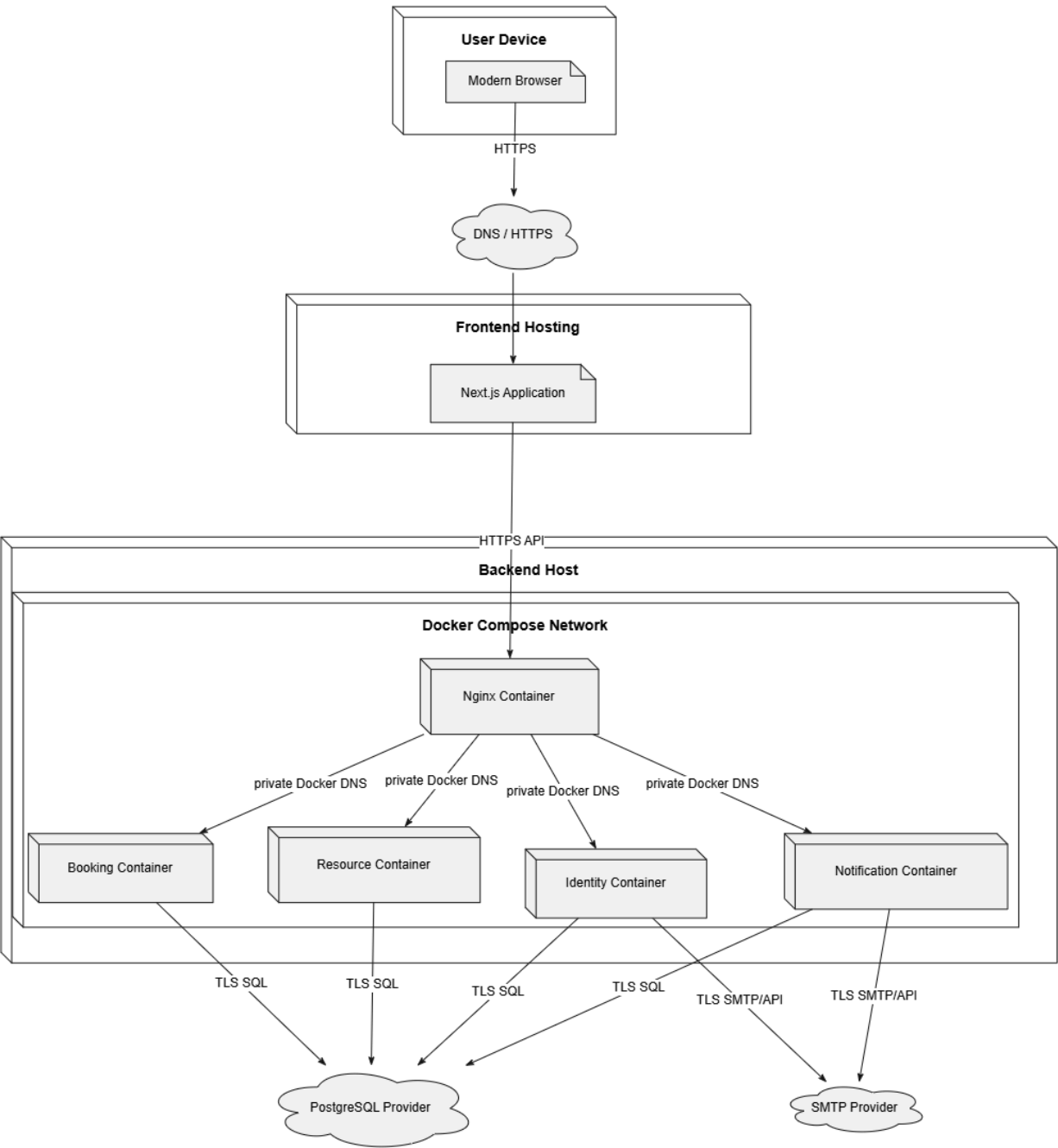


Figure 22. Initial Production Deployment

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

7.1 Deployment Security Boundaries

ResourceHive applies the following deployment-security boundaries:

- The browser never receives database credentials or connects directly to PostgreSQL.
- Only Nginx is publicly reachable on the backend host.
- Backend-service ports remain private to the Docker network.
- Public traffic uses HTTPS, while database and email-provider connections use encrypted transport.
- JWT secrets, OAuth credentials, SMTP credentials, and database URLs are supplied through secure environment configuration.
- Secrets are excluded from source code, container images, logs, and documentation.
- Health endpoints provide operational status without exposing credentials, stack traces, or tenant data.

7.2 Evolution

The initial deployment uses one instance of each backend service. If measured demand requires additional capacity, service containers may later be replicated behind Nginx.

Redis will be introduced for approved caching, notification pub/sub, rate limiting, or WebSocket coordination. It must not replace PostgreSQL as the authoritative source for bookings, points, tokens, notifications, or disputes.

8. Implementation View

ResourceHive is implemented as a pnpm monorepo containing the authenticated Next.js application, an Nginx API Gateway, four independently runnable NestJS services, shared internal packages, and committed database migrations. The Identity, Resource, Booking, and Notification Services each own their domain rules. Shared packages provide the Prisma client and common JWT authentication without transferring domain ownership between services.

8.1 Overview

The implementation uses layered dependencies. Pages and components in apps/web call the frontend API client, which sends public API requests through Nginx. Nginx selects the responsible backend service but does not perform domain authorization. NestJS controllers validate incoming requests, shared guards authenticate protected requests, application services enforce tenant and business rules, and repositories access PostgreSQL through @resourcehive/database. The Prisma schema and committed SQL migrations under db/schema define the shared persistent model and its integrity controls.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

8.2 Layers

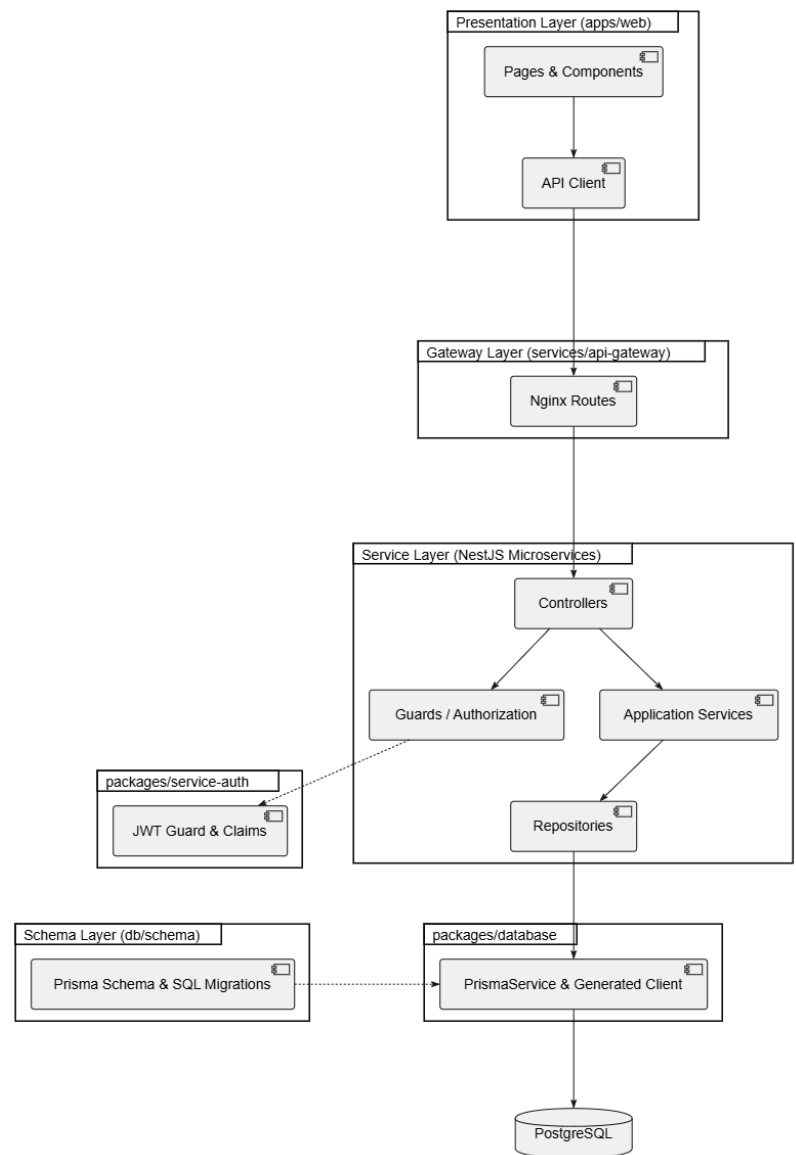


Figure 23. ResourceHive Implementation Layers

Figure 23 shows the implementation dependency path from the Next.js presentation layer through the Nginx gateway and NestJS service layer to PostgreSQL. Services reuse the shared authentication and database packages, while the Prisma schema and migrations define the generated database client. The frontend does not access PostgreSQL directly, and Nginx does not contain service business rules.

9. Data View

ResourceHive stores identity, tenant, resource, booking, points-ledger, notification, rating, and dispute information in one shared PostgreSQL database managed through committed Prisma migrations. Tenant boundaries are represented by organization relationships and root organization identifiers. The following parts of Figure 24 divide the complete data model into smaller readable domain views; repeated entities are reference points connecting those views.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

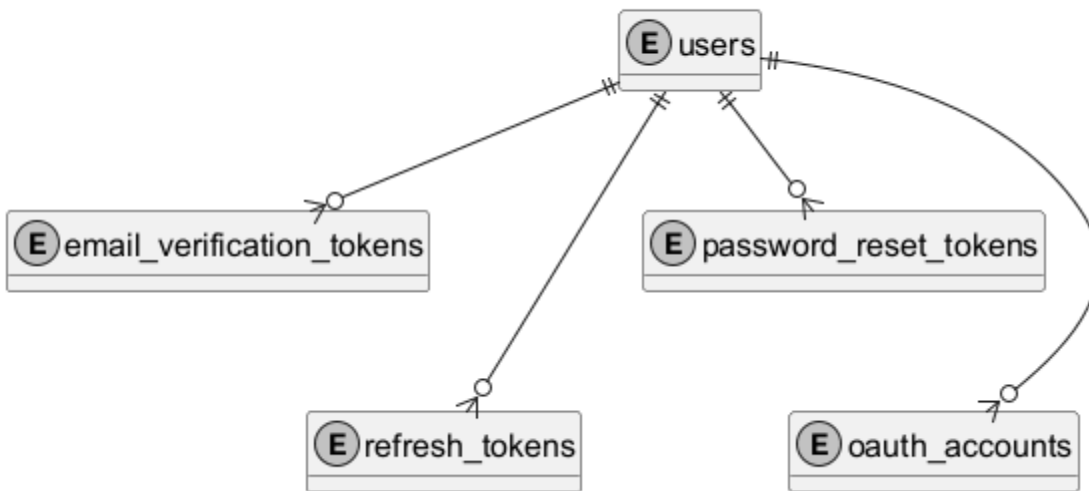


Figure 24(a). Identity and Authentication Data

Figure 24(a) shows the User table and the email-verification, password-reset, refresh-token, and optional OAuth records belonging to a user.

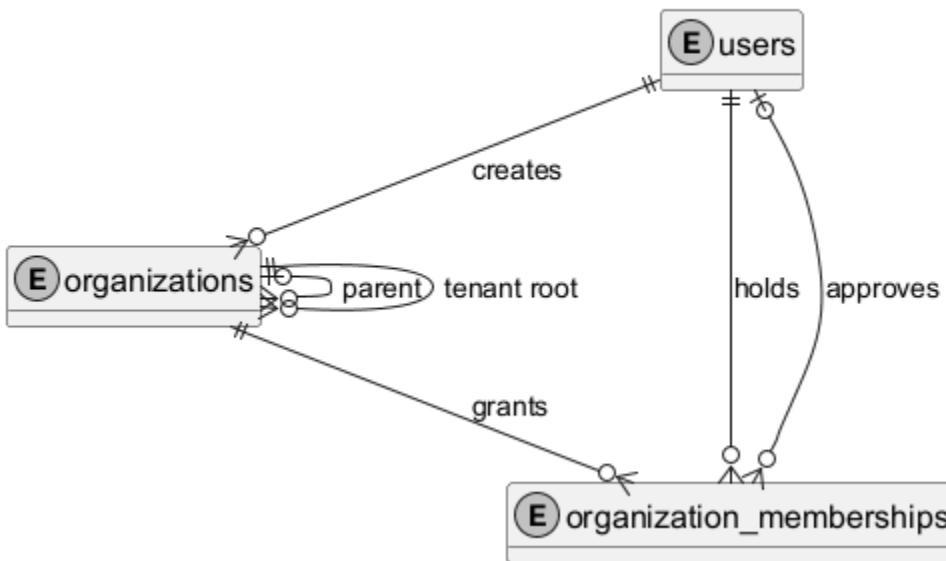


Figure 24(b). Tenant and Membership Data

Figure 24(b) shows organization creation, parent and root relationships, user memberships, and membership approval. These relationships provide the main tenant and organization access structure.

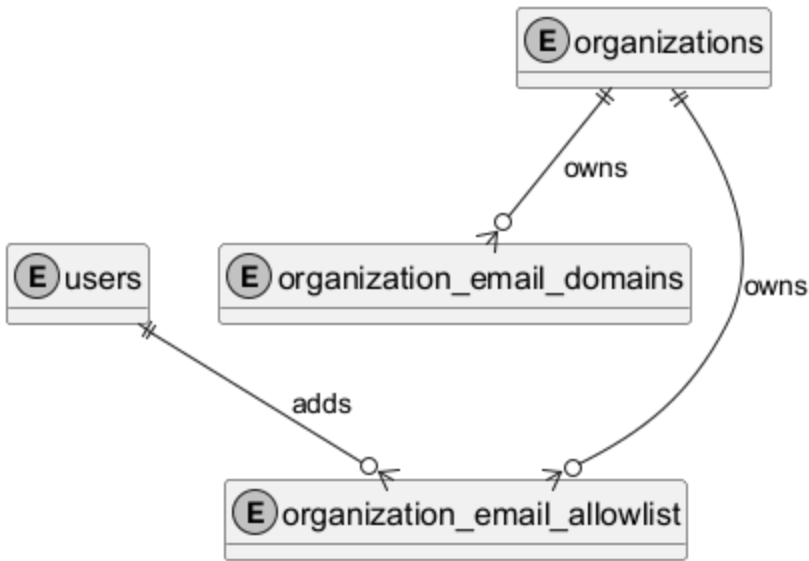


Figure 24(c). Registration Access Data

Figure 24(c) shows the root-organization email domains and organization email allowlist used during registration and membership processing. It also records the user who added each allowlist entry.

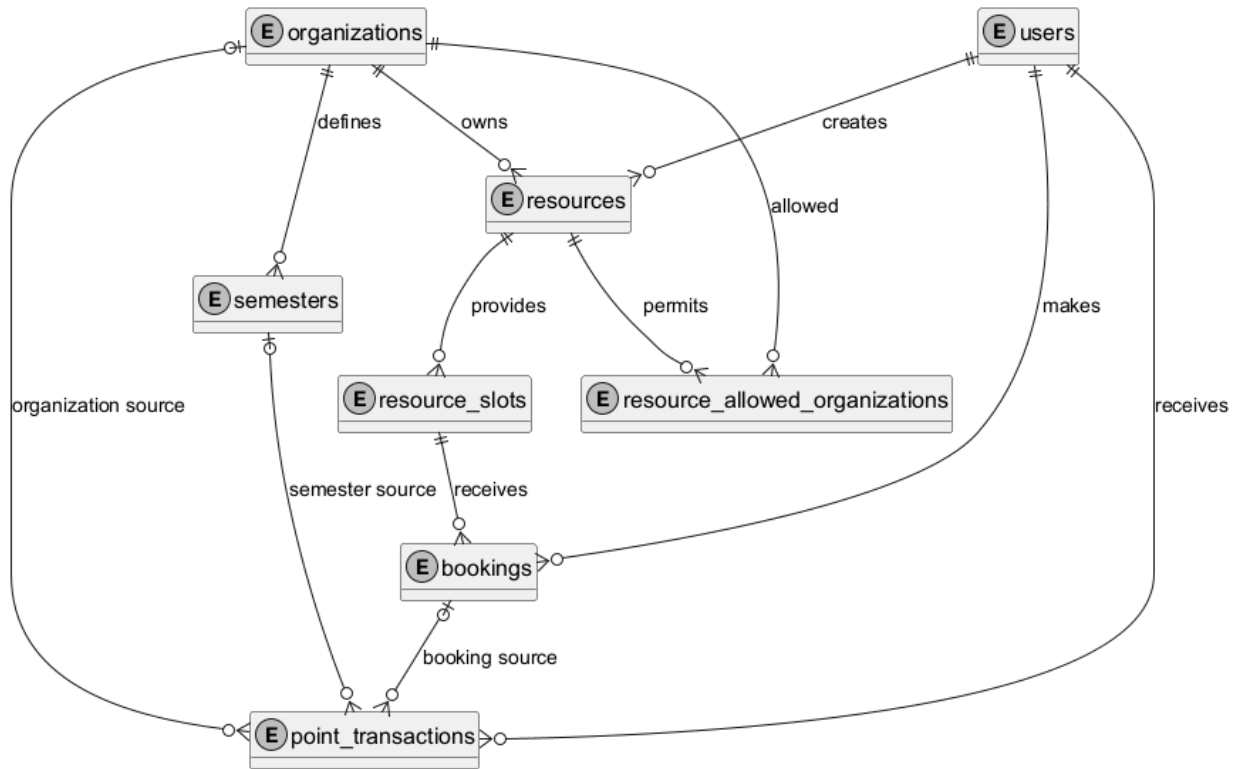


Figure 24(d). Resource, Booking, and Points Data

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

Figure 24(d) connects organizations, resources, allowed organizations, availability slots, bookings, semesters, and point transactions. Composite tenant relationships prevent cross-tenant resource access, while booking and point-source relationships preserve transaction history.

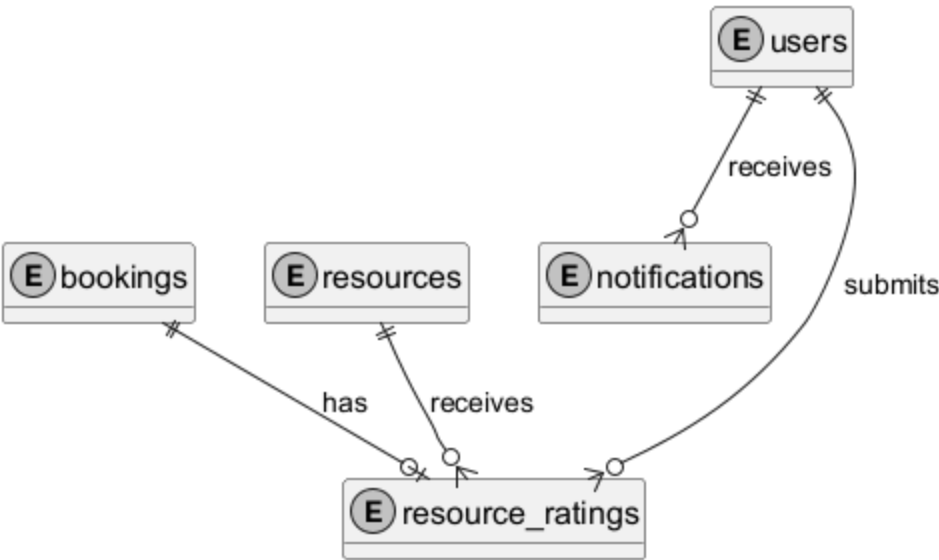


Figure 24(e). Notification and Rating Data

Figure 24(e) shows recipient-scoped notifications and ratings submitted by users for resources after an eligible completed booking.

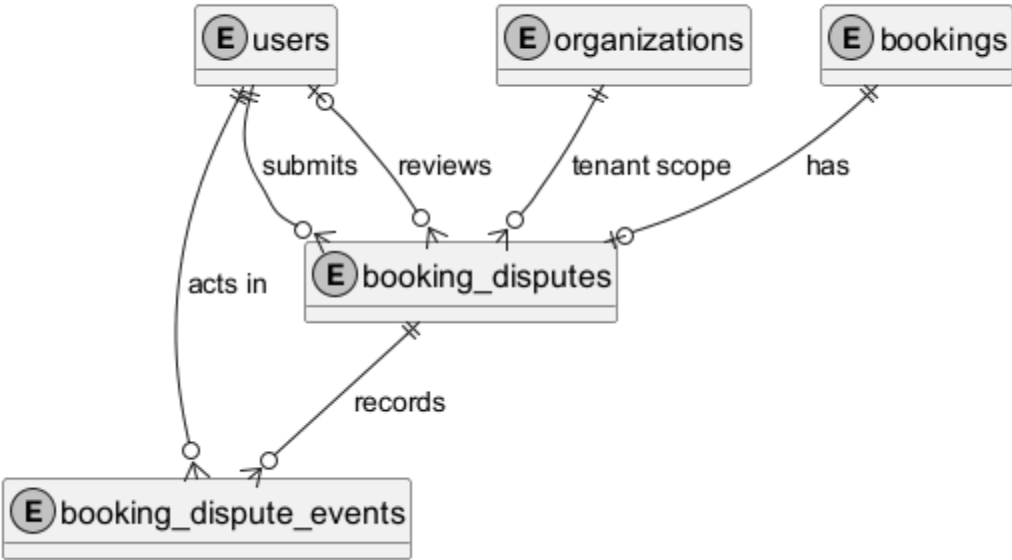


Figure 24(f). Booking Dispute Data

Figure 24(f) shows one optional dispute for a booking and the append-only event history recorded for that dispute. Users submit, review, and act in dispute events within the recorded root-organization scope.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

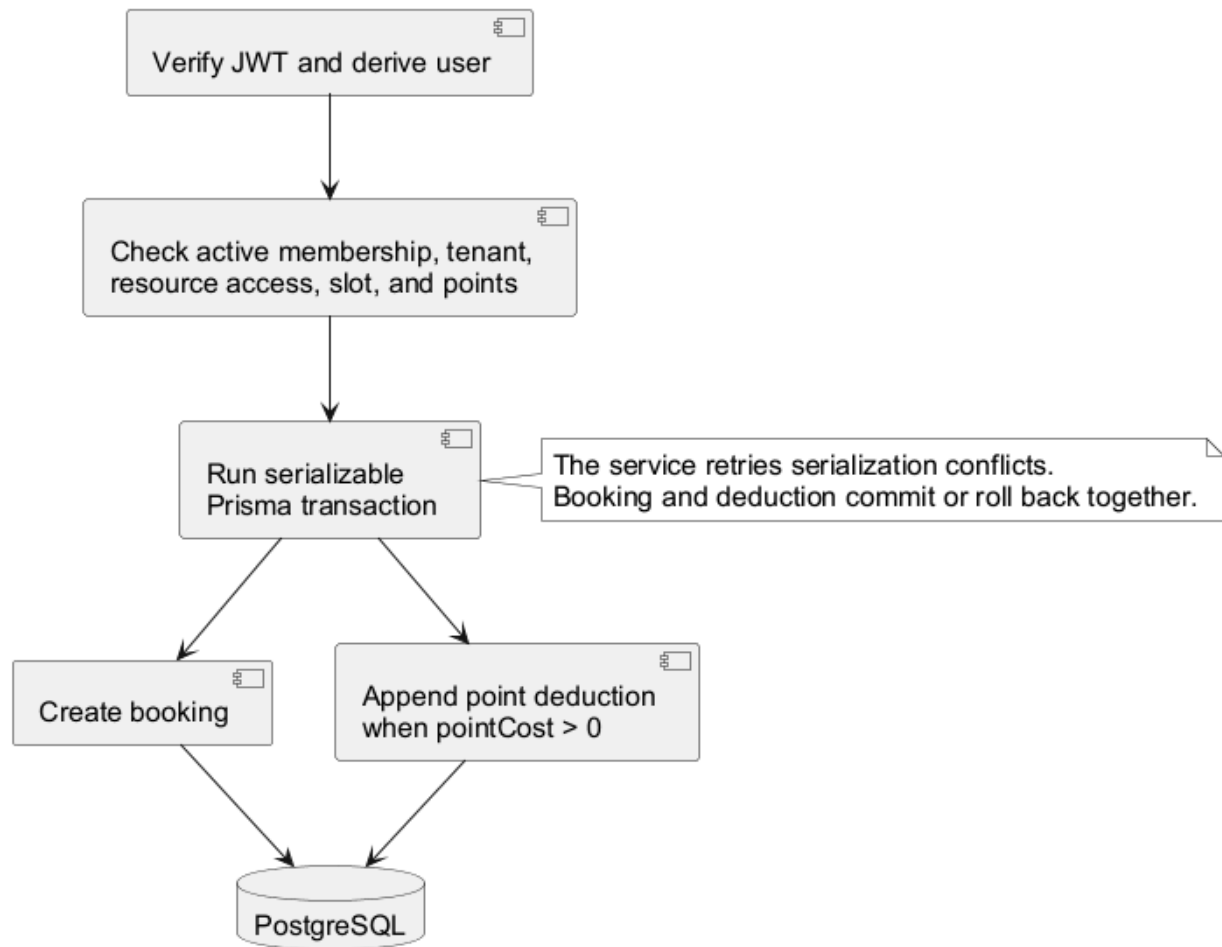


Figure 25(a). Atomic Booking and Concurrency Handling

Figure 25(a) follows the current Booking Service design. The service authenticates the user, validates membership, tenant, resource, slot, and point requirements, and then creates the booking inside one serializable Prisma transaction. A point deduction is appended in that transaction when the resource has a positive point cost. Serialization conflicts are retried, and all transaction writes commit or roll back together.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

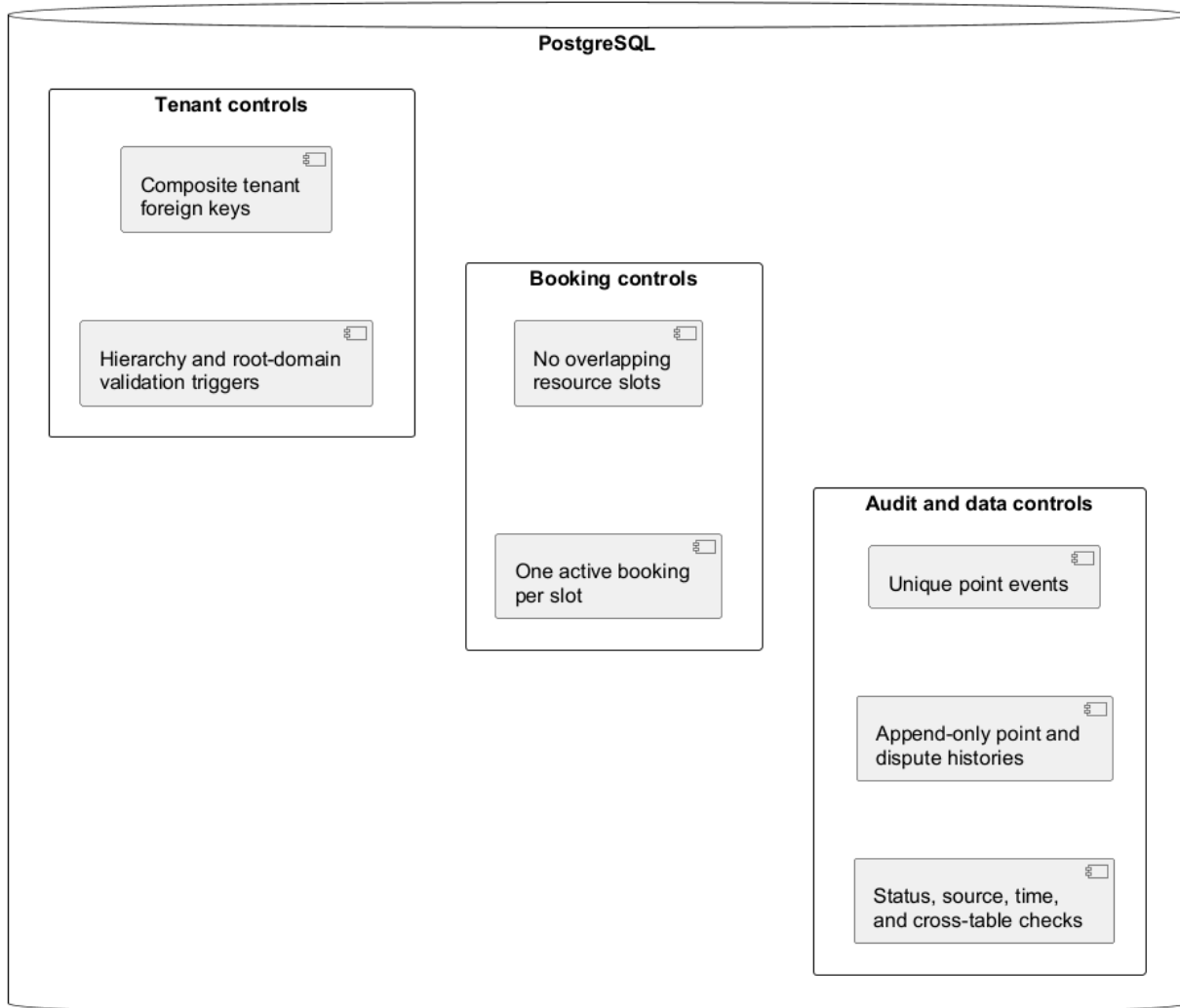


Figure 25(b). PostgreSQL Integrity Controls

Figure 25(b) groups the PostgreSQL constraints, indexes, and triggers that protect tenant relationships, organization hierarchy, resource-slot overlap, active-booking uniqueness, point-event uniqueness, append-only histories, timestamps, statuses, and cross-table relationships.

10. Size and Performance

The initial deployment uses one instance of each backend service and one shared PostgreSQL database. No numerical production target for concurrent users, throughput, response time, or availability has yet been approved; these values must be established through load testing before production acceptance. Performance-sensitive request paths avoid an Identity Service network call by verifying JWTs locally, use indexed tenant-scoped database queries, paginate list responses, and limit booking locks to the state required for correctness. Booking correctness and tenant isolation must not be weakened to improve response time. Stateless access-token validation and replaceable service containers allow later horizontal scaling when measured demand justifies it.

ResourceHive	Version: 1.0
Software Architecture Document	Date: 09 August 2026
RH-SAD-001	

11. Quality

ResourceHive protects reliability and data correctness through committed migrations, PostgreSQL constraints, database transactions, controlled failure responses, health checks, and automated unit, service, database, and concurrent-booking tests. Security and privacy are supported by HTTPS, a single public API gateway, local JWT verification, service-local authorization, tenant-scoped queries, hashed passwords and tokens, secure cookies, and environment-based secret configuration. Clear service ownership, shared technical packages, documented APIs, and continuous integration improve maintainability. Provider-neutral PostgreSQL, email, hosting, and deployment configuration improves portability, while consistent shadcn-based interfaces, meaningful validation messages, semantic controls, and keyboard support improve usability and accessibility.

12. References

- [1] ISO/IEC/IEEE, “Software, systems and enterprise Architecture description,” ISO/IEC/IEEE 42010:2022, 2022. [Online]. Available: <https://www.iso.org/standard/74393.html>. [Accessed: 08-Aug-2026].
- [2] Object Management Group, “Unified Modeling Language Specification, Version 2.5.1.” [Online]. Available: <https://www.omg.org/spec/UML/>. [Accessed: 08-Aug-2026].
- [3] draw.io, “draw.io Diagramming Tool.” [Online]. Available: <https://www.drawio.com/>. [Accessed: 08-Aug-2026].
- [4] PlantUML, “PlantUML Documentation.” [Online]. Available: <https://plantuml.com/>. [Accessed: 08-Aug-2026].
- [5] Vercel, “Next.js Documentation.” [Online]. Available: <https://nextjs.org/docs>. [Accessed: 29-Jul-2026].
- [6] NestJS, “NestJS Documentation.” [Online]. Available: <https://docs.nestjs.com/>. [Accessed: 29-Jul-2026].
- [7] PostgreSQL Global Development Group, “PostgreSQL 15 Documentation.” [Online]. Available: <https://www.postgresql.org/docs/15/>. [Accessed: 29-Jul-2026].
- [8] Prisma, “Prisma ORM Documentation.” [Online]. Available: <https://www.prisma.io/docs/orm>. [Accessed: 29-Jul-2026].
- [9] Docker, “Docker Compose Documentation.” [Online]. Available: <https://docs.docker.com/compose/>. [Accessed: 29-Jul-2026].
- [10] Nginx, “Nginx Documentation.” [Online]. Available: <https://nginx.org/en/docs/>. [Accessed: 29-Jul-2026].
- [11] OpenAPI Initiative, “OpenAPI Specification.” [Online]. Available: <https://spec.openapis.org/oas/latest.html>. [Accessed: 29-Jul-2026].
- [12] GitHub, “GitHub Actions Documentation.” [Online]. Available: <https://docs.github.com/en/actions>. [Accessed: 29-Jul-2026].
- [13] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, May 2015. [Online]. Available: <https://www.rfc-editor.org/info/rfc7519>. [Accessed: 29-Jul-2026].
- [14] World Wide Web Consortium, “Web Content Accessibility Guidelines (WCAG) 2.2.” [Online]. Available: <https://www.w3.org/TR/WCAG22/>. [Accessed: 29-Jul-2026].